

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ – VẬT LÝ KỸ THUẬT
BỘ MÔN VẬT LÝ HẠT NHÂN



KHOÁ LUẬN TỐT NGHIỆP ĐẠI HỌC

TÊN ĐỀ TÀI

**TỐI ƯU CODE NHÚNG CHO BỘ ĐIỆN TỬ CỦA HỆ ĐO LIỀU
PHÓNG XẠ VÀ NÂNG CẤP WEB HIỂN THỊ THEO THỜI
GIAN THỰC**

**SVTH: DƯƠNG NGỌC THÀNH
CBHD: PGS.TS. VÕ HỒNG HẢI**

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ – VẬT LÝ KỸ THUẬT
BỘ MÔN VẬT LÝ HẠT NHÂN



KHOÁ LUẬN TỐT NGHIỆP ĐẠI HỌC

TÊN ĐỀ TÀI

**TỐI ƯU CODE NHÚNG CHO BỘ ĐIỆN TỬ CỦA HỆ ĐO
LIỀU PHÓNG XẠ VÀ NÂNG CẤP WEB HIỂN THỊ THEO
THỜI GIAN THỰC**

SVTH: DƯƠNG NGỌC THÀNH
CBHD: PGS.TS. VÕ HỒNG HẢI

TP. Hồ Chí Minh, Năm 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành đến Ban Giám hiệu Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM cùng Quý Thầy Cô Khoa Vật lý - Vật lý Kỹ thuật đã tận tình giảng dạy và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập tại trường.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc nhất đến **PGS.TS. Võ Hồng Hải**. Thầy là người đã trực tiếp hướng dẫn, định hướng đề tài và tận tình chỉ bảo em từ những bước đi đầu tiên. Sự kiên nhẫn và những lời khuyên chuyên môn quý báu của Thầy đã giúp em vượt qua nhiều khó khăn để hoàn thiện khóa luận này.

Em xin gửi lời cảm ơn trân trọng đến **Thầy Võ Hoàng Nguyên** đã dành thời gian quý báu để đọc, đánh giá và đưa ra những ý kiến nhận xét xác đáng, giúp em nhận ra những thiếu sót để hoàn thiện khóa luận cũng như tư duy chuyên môn của mình.

Một lời tri ân đặc biệt và đầy xúc động em xin gửi đến **Cô Hoàng Thị Kiều Trang**. Trong những giai đoạn chông chênh nhất, lá thư an ủi mà Cô gửi đến thực sự là nguồn động viên ấm áp và duy nhất mà em nhận được lúc bấy giờ. Đồng thời, em xin khắc ghi ân tình của **Thầy Sang và Thầy Chiến**, hai thầy đã không quản ngại đường xa đến tận nhà để thăm hỏi, động viên tinh thần khi em gặp biến cố.

Bên cạnh đó, mình xin gửi lời cảm ơn thân thương đến **các bạn lớp 21VLHN và 22VLHN**. Cảm ơn các bạn đã không ngần ngại đồng hành, chia sẻ và tiếp thêm năng lượng tích cực, giúp mình vượt qua những khoảnh khắc buồn chán và chông chênh nhất trong cuộc sống học đường.

Cuối cùng, em muốn gửi lời xin lỗi và biết ơn chân thành nhất đến Quý Thầy Cô Bộ môn và Khoa. Em tự nhận thấy bản thân đã là một sinh viên "đặc biệt" theo cách không mong muốn, hay "báo thủ" và gây ra không ít phiền toái, lo lắng cho Thầy Cô, nhất là trong khoảng thời gian em buộc phải tạm ngưng việc học để điều trị bệnh. Em xin cảm ơn sự bao dung, kiên nhẫn và hỗ trợ không ngừng nghỉ của Thầy Cô, đã không bỏ rơi em lại phía sau và giúp em hoàn thành chặng đường đại học đầy chông gai này.

Kính chúc Quý Thầy Cô cùng các bạn luôn dồi dào sức khỏe và hạnh phúc.

Em xin chân thành cảm ơn.

Thành phố Hồ Chí Minh, tháng 01 năm 2026

Sinh viên thực hiện

Dương Ngọc Thành

MỤC LỤC

LỜI CẢM ƠN.....	ii
MỤC LỤC	iv
DANH MỤC KÝ HIỆU, CHỮ VIẾT TẮT	vii
DANH MỤC BẢNG BIỂU	viii
DANH MỤC HÌNH VẼ, ĐỒ THỊ.....	ix
DANH MỤC CODE NHÚNG	ix
LỜI MỞ ĐẦU.....	x
CHƯƠNG 1 – TỔNG QUAN, CƠ SỞ LÝ THUYẾT VÀ THIẾT KẾ HỆ THỐNG	11
1.1 Tổng quan về đề tài nghiên cứu	11
1.1.1 Đặt vấn đề về tính cấp thiết của đề tài	11
1.1.2 Mục tiêu nghiên cứu.....	11
1.1.2.1 Mục tiêu tối ưu hoá phần cứng	11
1.1.2.2 Mục tiêu nâng cấp phần mềm và nền tảng Cloud.....	12
1.2 Kỹ thuật lập trình bất đồng bộ và các dịch vụ AWS	12
1.2.1 Kỹ thuật lập trình bất đồng bộ Non-blocking I/O trên vi điều khiển.....	12
1.2.2 Cơ chế ngắt (Interrupt) trong ghi nhận trigger.....	12
1.2.3 Kiến trúc Serverless và các dịch vụ AWS mở rộng.....	12
1.2.4 Ứng dụng trí tuệ nhân tạo trong phân tích dữ liệu IoT	13
1.3 Phân tích hiện trạng và thiết kế giải pháp tối ưu	13
1.3.1 Phân tích các hạn chế kỹ thuật của hệ thống tiền nhiệm.....	13
1.3.2 Thiết kế luồng hoạt động tổng thể	13
1.3.3 Thiết kế giải pháp phần cứng mở rộng	14
CHƯƠNG 2 – THỰC HIỆN TỐI ƯU VÀ NÂNG CẤP HỆ THỐNG	16
2.1 Sơ đồ hệ thống quan trắc thời gian thực trực tuyến	16
2.2 Tối ưu Code nhúng (Firmware).....	16
2.2.1 Nâng cấp trạm quan trắc	17
2.2.2 Xây dựng hiển thị và cảnh báo trên màn hình LCD	18
2.3 Xây dựng backend serverless kiến trúc microservices trên AWS.....	20
2.3.1 Lập trình AWS Lambda xử lý dữ liệu	20
2.3.1.1 Quy trình tiếp nhận và phân luồng dữ liệu	20
2.3.1.2 Tổng hợp dữ liệu định kỳ và tối ưu chi phí vận hành.....	21
2.3.2 Lập trình API quản trị và điều khiển thiết bị	22

2.3.2.1 Cơ chế cầu nối điều khiển (HTTPS to MQTT Bridge)	23
2.3.2.2 Cơ chế giám sát trạng thái thực	23
2.3.3 Triển khai AI Chatbot và hệ thống phân tích thông minh	24
2.3.3.1 Thiết kế bộ định tuyến thông minh với Prompt Engineering.....	24
2.3.3.2 Cơ chế gọi hàm chéo	24
2.3.3.3 Tính năng lưu trữ và quản lý tri thức.....	25
2.3.4 Xây dựng quy trình CI/CD và quản lý phiên bản tập trung.....	25
2.3.4.1 Quản lý đăng nhập và kiểm tra quyền quản trị hệ thống	26
2.3.4.2 Quản lý vòng đời firmware và cơ chế OTA qua CDN	26
2.3.4.3 Tự động hoá triển khai Web với AWS Amplify.....	26
2.3.5 Tích hợp quản lý phân quyền và lưu trữ cá nhân.....	27
2.4 Thiết kế và cấu trúc hệ thống giao diện Web	28
CHƯƠNG 3 – KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ HIỆU NĂNG	29
3.1 Đánh giá hoạt động của hệ quan trắc phóng xạ thời gian thực	29
3.1.1 Kiểm tra dữ liệu và kết nối.....	29
3.1.2 Kiểm tra hệ thống tự kết internet.	29
3.1.3 Kiểm tra hiển thị và cảnh báo trên LCD	30
3.1.4 Kiểm tra dữ liệu được gửi lên AWS	31
3.2 Triển khai giao diện website cho hệ đo	31
3.2.1 Kết quả trung tâm giám sát quan trắc thời gian thực	31
3.2.2 Kết quả trang trạng thái và nhật ký hệ thống	32
3.2.3 Kết quả trang phân tích dữ liệu	34
3.3 Triển khai giao diện website hỗ trợ cho quản lý	36
3.3.1 Trang giao diện xác thực và bảo mật	36
3.3.2 Trang quản trị hệ thống và các chức năng điều khiển.....	36
3.4 Đánh giá hiệu quả kinh tế và tối ưu chi phí vận hành	38
3.4.1 Tổng quan và phân rã chi phí hạ tầng	38
3.4.2 Đánh giá mức độ sử dụng tài nguyên và tác động môi trường	39
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN MỞ RỘNG	41
Kết luận	41
Hướng phát triển mở rộng hệ thống	41
TÀI LIỆU THAM KHẢO	42
PHỤ LỤC 1 - MÃ NGUỒN CÁC TRẠM QUAN TRẮC PHÓNG XẠ.....	43

PHỤ LỤC 2 - CÁC FILE SECRETS.H	46
--------------------------------------	----

DANH MỤC KÝ HIỆU, CHỮ VIẾT TẮT

Chữ viết tắt	Tiếng Anh	Tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
ARN	Amazon Resource Names	Tên xác định tài nguyên Amazon
AWS	Amazon Web Services	Các dịch vụ web Amazon
BE	Back-end	Lập trình phía máy chủ
CPS	Counter Per Second	Số đếm trên giây
FE	Front-end	Lập trình giao diện
FSM	Finite State Machine	Máy trạng thái
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
LAN	Local Area Network	Mạng nội bộ
LLM	Large Language Model	Mô hình ngôn ngữ lớn
OTA	Over The Air	Cập nhật không dây
WAN	Wide Area Network	Mạng diện rộng
WDT	Watchdog Timer	Bộ đếm thời gian giám sát
μSv	Micro Sievert	Micro Sievert

DANH MỤC BẢNG BIỂU

Bảng 2.1. Bảng cấu hình chân pin map.....	18
--	----

DANH MỤC HÌNH VẼ, ĐỒ THỊ

Hình 1.1. Tổng quan 06 luồng hoạt động của hệ thống	14
Hình 2.1. Sơ đồ hệ thống quan trắc thời gian thực trực tuyến	16
Hình 2.2. Sơ đồ đầu nối chi tiết trạm 1 và trạm 4	17
Hình 2.3. Hình ảnh thực tế trạm 4 hoạt động với 2 nguồn.....	18
Hình 2.4. Sơ đồ luồng xử lý của hàm display()	19
Hình 2.5. Giải thuật tổng hợp dữ liệu và phân luồng dữ liệu.....	22
Hình 2.6. Sơ đồ luồng dữ liệu của AI Chatbot.....	25
Hình 2.7. Sơ đồ luồng CI/CD và OTA tổng hợp	27
Hình 2.8. Sơ đồ quản lý và phê duyệt người dùng.....	28
Hình 2.9. Sơ đồ cấu trúc Website theo quyền Admin, Khách và thành viên.....	28
Hình 3.1. Nhật ký Serial Monitor hoạt động đa nhiệm.....	29
Hình 3.2. Nhật ký ghi nhận hệ thống tự reset khi mất mạng sau 70 giây	30
Hình 3.3. Hình ảnh dữ liệu thực tế AWS nhận được từ JSON của trạm 3	31
Hình 3.4. Hình ảnh thực tế trạm 1 và 4 ở trạng thái bình thường.....	Error! Bookmark not defined.
Hình 3.5. Hình ảnh thực tế cảnh báo phóng xạ của trạm 1 và 4	Error! Bookmark not defined.
Hình 3.6. Giao diện đăng nhập của hệ thống	Error! Bookmark not defined.
Hình 3.7. Giao diện trung tâm giám sát thực tế trên Web	32
Hình 3.8. Hình ảnh thực tế về trạng thái hệ thống ghi nhận 18 giờ ngày 10/1/2026	33
Hình 3.9. Nhật ký hệ thống ghi nhận cảnh báo.....	33
Hình 3.10. Giao diện thiết lập tham số truy vấn dữ liệu	34
Hình 3.11. Biểu đồ phân tích xu hướng nhiệt độ và mật độ phóng xạ.....	35
Hình 3.12. Bảng phân tích thống kê chi tiết của 4 trạm.....	35
Hình 3.13. Bảng giá trị csv khi trích xuất dữ liệu từ trang phân tích.....	36
Hình 3.14. Giao diện tổng quan và quản lý người dùng	37
Hình 3.15. Giao diện điều khiển trạm và cấu hình chi tiết.....	37
Hình 3.16. Kho lưu trữ báo cáo và tài liệu của đề tài.....	38
Hình 3.17. Tổng quan chi phí và biểu đồ phân ra dịch vụ	39
Hình 3.18. Báo cáo sử dụng Free Tier và khí thải carbon	39

DANH MỤC CODE NHÚNG

Code 2.1. Đoạn code cấu trúc mô-đun riêng lẻ file aws_glitch.ino	17
--	----

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ Internet of Things (IoT) đang phát triển mạnh mẽ và đi sâu vào mọi lĩnh vực của đời sống, việc ứng dụng các giải pháp giám sát trực tuyến vào công tác an toàn bức xạ là một nhu cầu cấp thiết. Xuất phát từ thực tiễn đó, cùng với mong muốn khắc phục những hạn chế về độ trễ và khả năng xử lý của các hệ thống cũ, tôi đã thực hiện đề tài **“Tối ưu code nhúng cho bộ điện tử của hệ đo liều phóng xạ và nâng cấp Web hiển thị theo thời gian thực”**.

Bố cục của khóa luận được chia thành 3 chương chính với nội dung cụ thể như sau:

Chương 1: Tổng quan, cơ sở lý thuyết và thiết kế hệ thống. Chương này trình bày các kiến thức nền tảng về vật lý hạt nhân, nguyên lý hoạt động của ống đếm Geiger-Muller và các thách thức trong đo lường phóng xạ. Đồng thời, chương cũng cung cấp cái nhìn tổng quan về công nghệ IoT, kiến trúc vi điều khiển ESP32 với hệ điều hành FreeRTOS và các dịch vụ điện toán đám mây (AWS IoT Core, Lambda, DynamoDB) được sử dụng làm nền tảng cho hệ thống.

Chương 2: Thực hiện tối ưu và nâng cấp hệ thống. Đây là chương trọng tâm của đề tài, đi sâu vào các giải pháp kỹ thuật nhằm giải quyết bài toán tối ưu hóa. Nội dung bao gồm quy trình thiết kế phần cứng, chiến lược xây dựng Firmware đa nhiệm để xử lý tín hiệu xung nhiễu và truyền tin song song. Bên cạnh đó, chương cũng mô tả chi tiết quá trình xây dựng kiến trúc Serverless trên AWS và phát triển giao diện Dashboard tương tác hai chiều.

Chương 3: Kết quả thực nghiệm và đánh giá hiệu năng. Trình bày kết quả vận hành thực tế của hệ thống trọn vẹn từ thiết bị biên đến giao diện người dùng. Các nội dung chính bao gồm đánh giá độ ổn định của thiết bị, độ trễ của hệ thống cảnh báo, và phân tích hiệu quả kinh tế thông qua bài toán tối ưu chi phí vận hành trên nền tảng đám mây.

Cuối cùng là phần Kết luận và Hướng phát triển, tổng kết những đóng góp chính của đề tài và đề xuất các giải pháp mở rộng nhằm nâng cao tính ứng dụng của sản phẩm trong tương lai.

CHƯƠNG 1 – TỔNG QUAN, CƠ SỞ LÝ THUYẾT VÀ THIẾT KẾ HỆ THỐNG

1.1 Tổng quan về đề tài nghiên cứu

1.1.1 Đặt vấn đề về tính cấp thiết của đề tài

Trong kỷ nguyên công nghiệp 4.0, vấn đề an toàn bức xạ tại các cơ sở y tế và các khu công nghiệp sử dụng nguồn phóng xạ đang trở thành mối quan tâm hàng đầu. Mặc dù các thiết bị đo phóng xạ cầm tay truyền thống đảm bảo độ chính xác cơ bản nhưng thường bộc lộ nhiều hạn chế trong mô hình quản lý hiện đại, chúng hoạt động độc lập, thiếu khả năng cảnh báo tức thời từ xa và khó khăn trong việc tổng hợp dữ liệu lịch sử.

Tiếp nối hướng nghiên cứu về hệ thống quan trắc phóng xạ trực tuyến của đề tài khoá luận trước “*Thiết lập hệ thống truyền dữ liệu AWS thời gian thực trực tuyến ứng dụng cho hệ quan trắc phóng xạ*” [1] đã bước đầu xây dựng được mô hình truyền dữ liệu cảm biến lên hệ thống AWS. Tuy nhiên, qua quá trình vận hành thực nghiệm và phân tích mã nguồn nhận thấy hệ thống tiền nhiệm tồn tại những nhược điểm kỹ thuật ảnh hưởng đến tính ổn định:

- **Hiện tượng nghẽn mạch Blocking:** Firmware cũ sử dụng hàm tạo tín hiệu trễ “delay()” và xử lý tuần tự. Khi mất kết nối mạng hoặc thực hiện tác vụ nặng, vi điều khiển MCU bị treo dẫn đến việc bỏ sót các xung phóng xạ đi qua.
- **Thiếu cơ chế phục hồi kết nối Reconnection Strategy:** Thiết bị không có thuật toán tự động kết nối lại thông minh khi sự cố kết nối internet trở nên gián đoạn, chấp chờn và cần thực hiện khởi động thủ công để khắc phục tình trạng.

Xuất phát từ những điểm yếu của hệ thống cùng với mục tiêu tối ưu và nâng cấp, đề tài “*Tối ưu code nhúng cho bộ điện tử của hệ đo liều phóng xạ và nâng cấp web hiển thị theo thời gian thực*” được thực hiện nhằm giải quyết triệt để bài toán ổn định của hệ thống và nâng tầm trải nghiệm cho người dùng bằng các công nghệ tiên tiến ở thời điểm hiện tại.

1.1.2 Mục tiêu nghiên cứu

1.1.2.1 Mục tiêu tối ưu hoá phần cứng

Chuyển đổi kiến trúc firmware sang mô hình bất đồng bộ Non-blocking I/O để đảm bảo khả năng xử lý đa nhiệm không ảnh hưởng đến các đơn nhiệm đang hoạt động.

Hiện thực hoá giải thuật “Glitch Handling” nhằm tự động phát hiện và xử lý các sự cố mất kết nối mạng mà không làm gián đoạn quá trình đo đạc.

Thực hiện tối ưu hoá code nhúng và mở rộng thử nghiệm lên 04 trạm quan trắc với sự đa dạng về phần cứng nhằm kiểm chứng mức độ tương thích của mã nguồn sau khi được tối ưu hoá.

1.1.2.2 Mục tiêu nâng cấp phần mềm và nền tảng Cloud

Xây dựng phần mềm về hạ tầng Backend BE theo kiến trúc serverless trên AWS để tối ưu hoá chi phí và nâng cao hiệu năng với các dịch vụ mà AWS có sẵn.

Phát triển Web Dashboard hiển thị biểu đồ theo thời gian thực nhằm đánh giá tình hình liều lượng phóng xạ đúng và theo sát tình hình thực tế.

Tích hợp trí tuệ nhân tạo AI sử dụng mô hình ngôn ngữ lớn - Large Language Model LLM để truy vấn dữ liệu phóng xạ khi có yêu cầu phân tích cơ bản hoặc truy vấn trạng thái trong khung chat với AI.

1.2 Kỹ thuật lập trình bất đồng bộ và các dịch vụ AWS

1.2.1 Kỹ thuật lập trình bất đồng bộ Non-blocking I/O trên vi điều khiển

Trong các hệ thống nhúng thời gian thực việc sử dụng hàm delay() là điều không nên vì nó lãng phí chu kỳ máy và chặn các tiến trình khác như việc đọc cảm biến và ghi nhận tín hiệu bị gián đoạn. Đề tài áp dụng kỹ thuật non-blocking dựa trên việc kiểm tra thời gian trôi qua “millis()” kết hợp với máy trạng thái Finite State Machine - FSM [2]. Điều này cho phép vi điều khiển thực hiện song song tác vụ nhưng quét hệ thống, cập nhật màn hình LCD và duy trì kết nối MQTT mà không bị chặn bởi tác vụ khác.

1.2.2 Cơ chế ngắt (Interrupt) trong ghi nhận trigger

Đầu dò Geiger-Muller tạo ra một xung điện cực ngắn từ vài micro giây đến vài trăm micro giây mỗi khi có bức xạ hạt nhân đi qua và ghi nhận. Việc sử dụng vòng lặp “polling” để kiểm tra chân tín hiệu dễ dẫn đến bỏ sót xung. Hệ thống sử dụng cơ chế ngắt-interrupt ngoài làm cho MCU sẽ tạm dừng các tác vụ hiện tại để tăng biến đếm ngay lập tức khi phát hiện sườn xung, đảm bảo độ chính xác tuyệt đối trong ghi nhận số xung (số đếm), có thể cho thông tin về tốc độ đếm CPS- Count Per Second hay CPM-Count Per Minute.

1.2.3 Kiến trúc Serverless và các dịch vụ AWS mở rộng

Thay vì duy trì một máy chủ ảo EC2 chạy liên tục gây tốn kém, đề tài sử dụng kiến trúc serverless với các dịch vụ cốt lõi:

- AWS IoT Core: Đóng vai trò Broker trung chuyển tin nhắn MQTT bảo mật (TSL/SSL).

- AWS Lambda: Các hàm được viết bằng python chỉ chạy khi có sự kiện giúp tiết kiệm chi phí vận hành tối đa.
- DynamoDB: Cơ sở dữ liệu NoSQL lưu trữ dữ liệu chi cấu trúc với tốc độ truy xuất cao.

1.2.4 Ứng dụng trí tuệ nhân tạo trong phân tích dữ liệu IoT

Đề tài tích hợp API mô hình ngôn ngữ lớn của Google Gemini 3 được sử dụng để phân tích dữ liệu trong trang “Phân tích dữ liệu - data-analysis.html” và trong khung chatbot AI nhằm phản hồi dữ liệu hoặc yêu cầu do người dùng cung cấp gửi đến hệ thống.

1.3 Phân tích hiện trạng và thiết kế giải pháp tối ưu

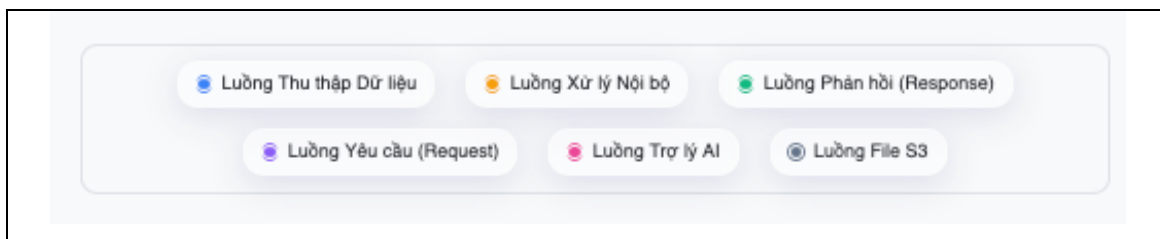
1.3.1 Phân tích các hạn chế kỹ thuật của hệ thống tiền nhiệm

Hệ thống quan trắc phóng xạ được phát triển trong giai đoạn 2024 hoạt động dựa trên mô hình xử lý tuần tự. Qua quá trình phân tích mã nguồn và thử nghiệm thực tế đã xác định được ba vấn đề cần được cải thiện và tối ưu cho đề tài hiện tại:

- **Vấn đề Blocking I/O trong firmware:** Hệ thống cũ sử dụng hàm delay() để định thời gian gửi dữ liệu và các vòng lặp while() để chờ kết nối WiFi/MQTT [3]. Điều này dẫn đến vi điều khiển buộc phải thực hiện lệnh chờ khi kết nối mạng WiFi bị lỗi làm cho CPU hoàn toàn bị khoá. Trong thời gian này, các ngắt (Interrupt) từ cảm biến của Geiger-Muller có thể bị bỏ qua hoặc không được xử lý kịp thời, dẫn đến sai số lớn trong việc đo đạc các sự kiện phóng xạ.
- **Thiếu cơ chế bộ đệm dữ liệu (Data Buffering):** Dữ liệu hệ thống ghi nhận được đều sẽ phải gửi lên hệ thống ngay lập tức trong điều kiện kết nối WiFi ổn định. Điều này dẫn đến việc nếu hệ thống không được kết nối WiFi hoặc kết nối không ổn định khiến cho dữ liệu đo đạc tại thời điểm đó sẽ bị mất vĩnh viễn. Hệ thống không có khả năng lưu trữ tạm thời để có thể thực hiện gửi bù dữ liệu khi hệ thống mạng WiFi được khôi phục, tạo ra các khoảng trống trong biểu đồ giám sát thời gian thực.

1.3.2 Thiết kế luồng hoạt động tổng thể

Thay vì mô tả hệ thống theo các lớp chức năng tách biệt, đề tài tiếp cận theo hướng phân tích luồng dữ liệu. Dựa trên cơ chế vận hành thực tế của website và mã nguồn JavaScript “workflow.js”, hoạt động của hệ thống quan trắc phóng xạ IoT được chia thành 06 luồng xử lý độc lập nhưng hỗ trợ chặt chẽ, liên kết các luồng dữ liệu từ khi thu thập đến khi hiển thị và bảo trì.



Hình 1.1. Tổng quan 06 luồng hoạt động của hệ thống

- **Luồng thu thập dữ liệu** chịu trách nhiệm chuyển đổi tín hiệu vật lý thành dữ liệu số trên Cloud Server AWS [4]. Dữ liệu được đóng gói dưới dạng JSON nhằm mục đích tối ưu băng thông.
- **Luồng xử lý nội bộ** được thiết lập dùng để giải quyết bài toán liên quan đến hiệu năng khi dữ liệu tích lũy lớn dần theo thời gian.
- **Luồng phản hồi** thực hiện tương tác đưa dữ liệu đã xử lý đến người dùng thông qua giao diện trực quan Website.
- **Luồng yêu cầu** thực hiện khả năng điều khiển hai chiều (Bi-directional) cho phép người quản trị can thiệp vào hệ thống của phần cứng từ xa hoặc thực hiện các chức năng điều khiển trên hệ thống website.
- **Luồng trợ lý AI** đây là sự nâng cấp lớn nhất về công nghệ, nâng cấp từ hệ thống “quan trắc thụ động” sang “tư vấn chủ động”. Luồng hoạt động được thêm vào với mục đích tiếp cận đa dạng người dùng nếu được triển khai thực tế ở quy mô lớn.
- **Luồng GitHub và AWS Amplify (CI/CD Deployment Flow)** [5] thực hiện quy trình DevOps tự động hoá, đảm bảo tính ổn định và khả năng cập nhật nhanh chóng của ứng dụng Web. GitHub đóng vai trò tích hợp liên tục CI, là nơi chứa toàn bộ mã nguồn của website “iot-rad-monitor

1.3.3 Thiết kế giải pháp phần cứng mở rộng

Khác với phát triển của đề tài tiền nhiệm chỉ tập trung vào một cấu hình phần cứng duy nhất thì đối với đề tài này hướng đến việc xây dựng một mạng lưới quan trắc phân tán. Để có thể kiểm chứng được tính linh hoạt của kiến trúc phần mềm và khả năng mở rộng của hệ thống, thiết kế phần cứng được triển khai thành 03 biến thể cấu hình áp dụng cho 04 trạm quan trắc thực tế hoàn toàn khác biệt. Điểm chung duy nhất là các trạm đều sử dụng mạch giao tiếp cảm biến RadiationD Cajoe - kết nối với đầu dò Geiger Muller và cảm biến nhiệt độ - độ ẩm DHT11 để ghi nhận dữ liệu. Sự đa dạng hoá nằm ở công nghệ hiển thị và bộ xử

lý trung tâm của 04 trạm quan trắc phóng xạ. Bảng 1.1 là bảng cấu hình phần cứng và các thông số kỹ thuật cho việc xây dựng 04 trạm quan trắc phóng xạ

Bảng 1.1. Cấu hình phần cứng và thông số kỹ thuật của 04 trạm quan trắc

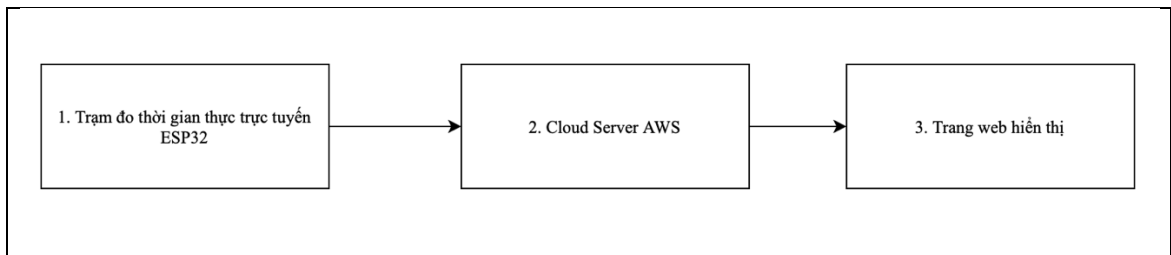
Đặc tính	Trạm 1 và trạm 4	trạm 2	Trạm 3
MCU (Vi điều khiển)	Diymore ESP32 dual-core [6]	ESP8266	ESP8266
Kiến trúc xử lý	Đa nhân	Đơn nhân	Đơn nhân
Công nghệ hiển thị	LCD 1.9” (Pixel màu)	LCD 1602 (Ký tự)	OLED 0.96” (Pixel đơn sắc)
Giao tiếp màn hình	SPI (Tốc độ cao)	I2C (module đệm)	I2C (on-board)
Cảm biến phóng xạ	Geiger Couter	Geiger Couter	Geiger Couter

CHƯƠNG 2 – THỰC HIỆN TỐI ƯU VÀ NÂNG CẤP HỆ THỐNG

2.1 Sơ đồ hệ thống quan trắc thời gian thực trực tuyến

Hệ quan trắc phóng xạ được thiết lập gồm 3 phần chính, hình 2.1:

1. Trạm đo phóng xạ (liều lượng) thời gian thực được thiết kế phần cứng gồm đầu dò GM, bộ giao tiếp nhúng, và hiển thị màn hình.
2. Hạ tầng cloud server AWS được xây dựng hoạt động chi tiết từ AWS IoT Core đến các hàm lambda dùng để xử lý dữ liệu tải lên và lưu trữ tại DynamoDB.
3. Trang web hiển thị sử dụng các ngôn ngữ lập trình front end HTML5, CSS3, JS.



Hình 2.1. Sơ đồ hệ thống quan trắc thời gian thực trực tuyến

2.2 Tối ưu Code nhúng (Firmware)

Trạm quan trắc phóng xạ thời gian thực được xây dựng dựa vào thiết bị phần cứng nhúng. Ở phát triển trước, một trong những hạn chế lớn nhất lập trình nguyên khối, tức là toàn bộ mã nguồn từ khai báo thư viện, định nghĩa biến cục, cấu hình mạng cho đến các xử lý cảm biến và giao thức MQTT đều được dồn chung vào một tệp tin duy nhất. Cách tiếp cận này khiến mã nguồn trở nên cồng kềnh, khó kiểm soát lỗi và gần như khó có thể mở rộng hoặc bảo trì khi số lượng tính năng mở rộng tăng lên. Chính vì sự hạn chế đó, đề tài này thực hiện việc tái cấu trúc toàn diện hệ thống firmware theo hướng tách riêng biệt thành các mô-đun chức năng riêng. Cụ thể,

- Chuyển đổi từ kiến trúc nguyên khối sang mô-đun riêng: Thiết lập tham số cấu hình ban đầu, xử lý MQTT, quản lý OTA.
- Chuẩn hoá quản lý cấu hình và biến toàn cục.
- Tối ưu hoá luồng xử lý dữ liệu.
- Tích hợp tính năng cập nhật không dây (OTA - Over The Air) [7]

Ở phiên bản tối ưu code được tinh gọn, như trình bày ở mã code 2.1, đóng vai trò như một người điều phối chỉ gọi các hàm chức năng đã được định nghĩa trong header “config_globals.h”. Điều này giúp luồng hoạt động của vi điều khiển trở nên tinh gọn và dễ hình dung.

Ở đó hàm config_globals.h chứa đựng các hàm con chuyên dụng, thực hiện các chức năng riêng biệt. Chi tiết về code config_globals.h được trình bày ở phụ lục 1.

```
#include "config_globals.h"

void setup()
{
    Init(); //khởi tạo phần cứng
    eeprom(); //cấu hình bộ nhớ
    arduinoOTA(); //cấu hình nạp không dây OTA
    connectAWS(); //kết nối AWS
}

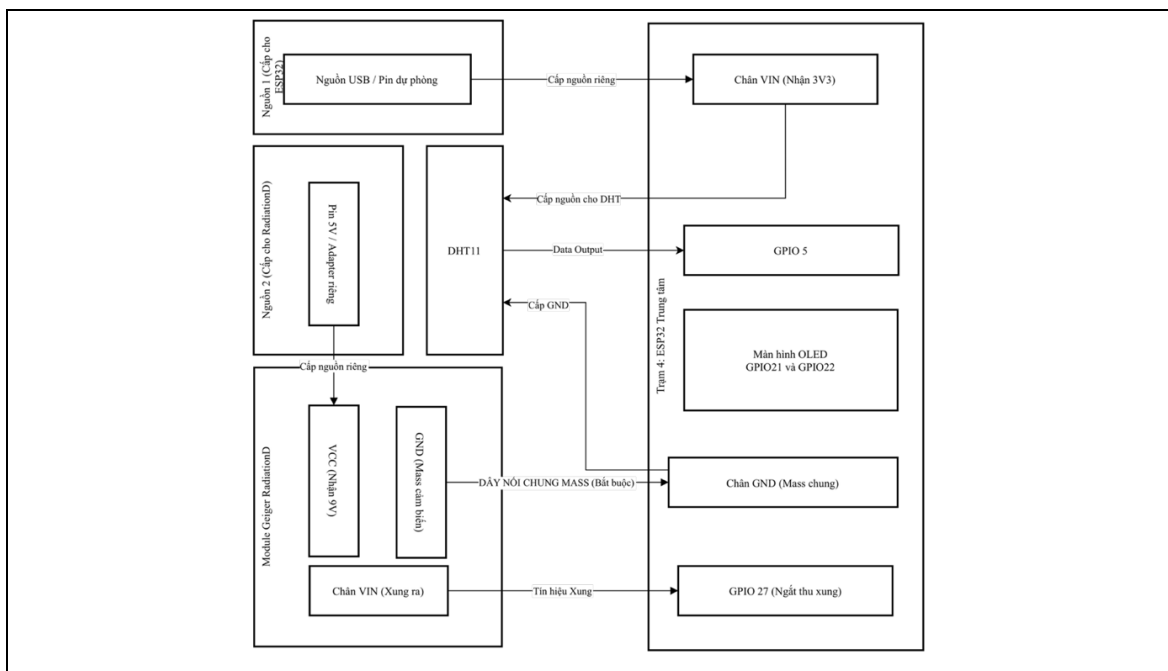
void loop()
{
    ArduinoOTA.handle(); //lắng nghe cập nhật
    readData(); //Đọc cảm biến và xử lý logic
    display(); //Hiển thị ra màn hình
}
```

Code 2.1. Đoạn code cấu trúc mô-đun riêng lẻ

2.2.1 Nâng cấp trạm quan trắc

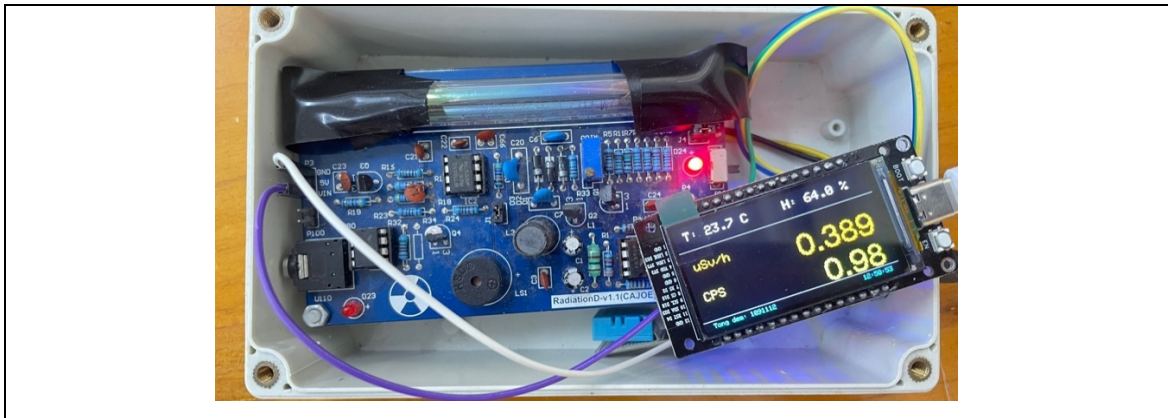
Trạm quan trắc có cấu hình phần cứng bao gồm ESP32 dual-core với màn hình hiển thị LCD 1.9”. Hệ thống sử dụng ESP32 dual-core để vừa ghi nhận tín hiệu từ đầu dò GM, vừa hiển thị lên màn hình LCD và vừa duy trì kết nối bảo mật với AWS IoT Core.

Hình 2.2 là sơ đồ đầu nối chi tiết giữa đầu dò GM, cảm biến nhiệt độ - độ ẩm với vi điều khiển ESP32 và chi tiết các chân kết nối ở bảng 2.1.



Hình 2.2. Sơ đồ đầu nối giữa đầu dò GM, cảm biến nhiệt độ/độ ẩm với ESP-32.

Hình 2.3 là hình ảnh thực tế về thiết bị phần cứng cho thiết kế một trạm, hiển thị các thông số nhận được từ đầu dò GM, cảm biến nhiệt độ - độ ẩm trên màn hình LCD 1.9”.



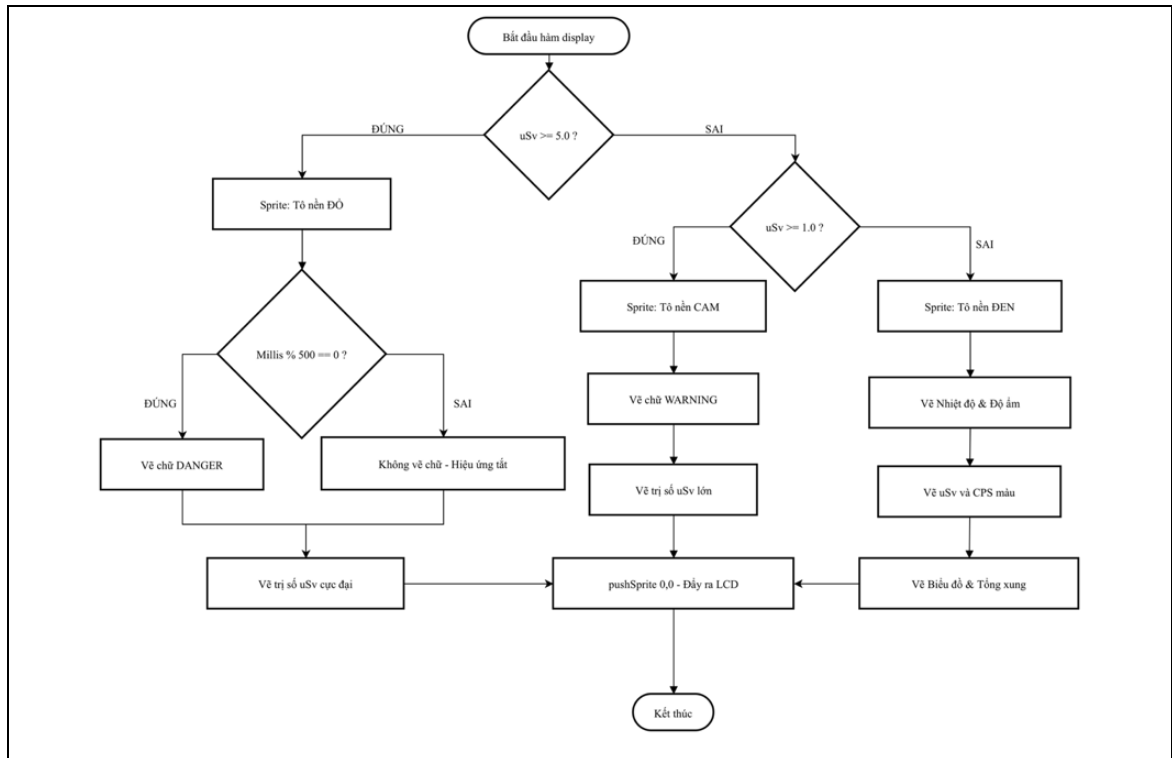
Hình 2.3. Hình ảnh thực tế của thiết bị

Bảng 2.1. Bảng cầu hình chân pin map

Thiết bị	Chân thiết bị	Nối	Ghi chú kỹ thuật
DHT11	VCC	ESP32 3V3	Đảm bảo dòng điện ổn định
	GND	ESP32 GND	Mass
	DATA	GPIO 5	Chân dữ liệu nhiệt độ, độ ẩm
LCD 1.9”	VCC/GND	ESP32 3V3/GND	Chung nguồn với DHT11
	I2C SDA	GPIO21	Cổng giao tiếp I2C
	I2C SCL	GPIO22	
Đầu dò GM	VCC	5V USB	Nguồn độc lập 5V
	GND	ESP32 GND	Mass chung ESP32 đồng bộ tín hiệu
	VIN	GPIO27	Chân ngắt đếm xung phóng xạ

2.2.2 Xây dựng hiển thị và cảnh báo trên màn hình LCD

Hệ thống trạm quan trắc thiết kế phần hiển thị trên màn hình LCD 1.9” kèm theo cảnh báo xung phóng xạ liên tục và thay đổi màu nền hiển thị của LCD.



Hình 2.4. Sơ đồ luồng xử lý của hàm display()

Dựa vào điểm mạnh vốn có sẵn của phần cứng ESP32, phần mềm firmware được khai thác tối đa về các hàm cảnh báo phóng xạ đa cấp độ. Hình 2.4 là giải thuật về cảnh báo được sử dụng trong mã nguồn với hàm display() phân chia rõ 3 cấp độ nguy hiểm bao gồm:

- **Bình thường (Normal):** Nền đen, hiển thị chi tiết các thông số nhận được từ phần cứng DHT11 và đầu dò GM như nhiệt độ, độ ẩm, μSv , cps, số đếm tích lũy và thời gian thực của trạm
- **Cảnh báo (Warning):** Logic được cài đặt sẵn trong mã nguồn với mức chuẩn `#define WARNING_THRESHOLD 1.0`. Tức là khi $\mu\text{Sv} \geq 1.0$ thì màn hình LCD sẽ chuyển toàn bộ sang màn hình cam kèm theo dòng chữ “WARNING” lớn nổi bật.
- **Nguy hiểm (Danger):** Với logic ngưỡng phóng xạ được cài đặt trong mã nguồn `#define DANGER_THRESHOLD 5.0` thì khi $\mu\text{Sv} \geq 5.0$ nền sẽ chuyển sang màn hình đỏ kèm theo dòng chữ “DANGER” nhấp nháy.

Đặc biệt hiệu ứng nhấp nháy được thực hiện bằng thuật toán bất đồng bộ Non-blocking dựa trên hàm “millis()” đảm bảo vi điều khiển không bị dừng lại trong việc hiển

thị màn hình cảnh báo hoặc màn hình nguy hiểm, giúp việc đo đạc và gửi dữ liệu lên AWS vẫn diễn ra liên tục ngay cả trong tình trạng báo động.

2.3 Xây dựng backend serverless kiến trúc microservices trên AWS

Một phát triển của đề tài so với hệ thống tiền nhiệm nằm ở việc tái cấu trúc toàn diện hạ tầng BE (Back End). Hệ thống cũ vận hành dựa theo một hàm xử lý đơn lẻ với nhiệm vụ nhận thông số JSON từ hệ thống và hiển thị dữ liệu thô sơ thì hệ thống mới đã được nâng cấp thành một kiến trúc microservices hoàn chỉnh trên nền tảng serverless AWS.

Hệ sinh thái BE hiện tại gồm 20 hàm AWS Lambda chuyên biệt, hoạt động độc lập nhưng liên kết chặt chẽ với nhau thông qua cơ chế hướng sự kiện event-driven. Sự phân tách không chỉ giải quyết triệt để bài toán nghẽn dữ liệu khi xử lý lượng dữ liệu lớn và còn tăng cường tính bảo mật và khả năng mở rộng của hệ thống. Cấu trúc BE được chia thành 04 phân hệ xử lý chính:

2.3.1 Lập trình AWS Lambda xử lý dữ liệu

Trong kiến trúc serverless, phân hệ xử lý dữ liệu đóng vai trò như một nhà máy số chịu trách nhiệm chuyển đổi hàng loạt tín hiệu thô từ các trạm quan trắc thành thông tin có giá trị sử dụng. Hệ thống không sử dụng một cơ sở dữ liệu đơn nhất mà áp dụng chiến lược đa hình lưu trữ trên nền tảng Amazon DynamoDB kết hợp với các hàm Lambda chuyên biệt cho từng tác vụ riêng lẻ.

2.3.1.1 Quy trình tiếp nhận và phân luồng dữ liệu

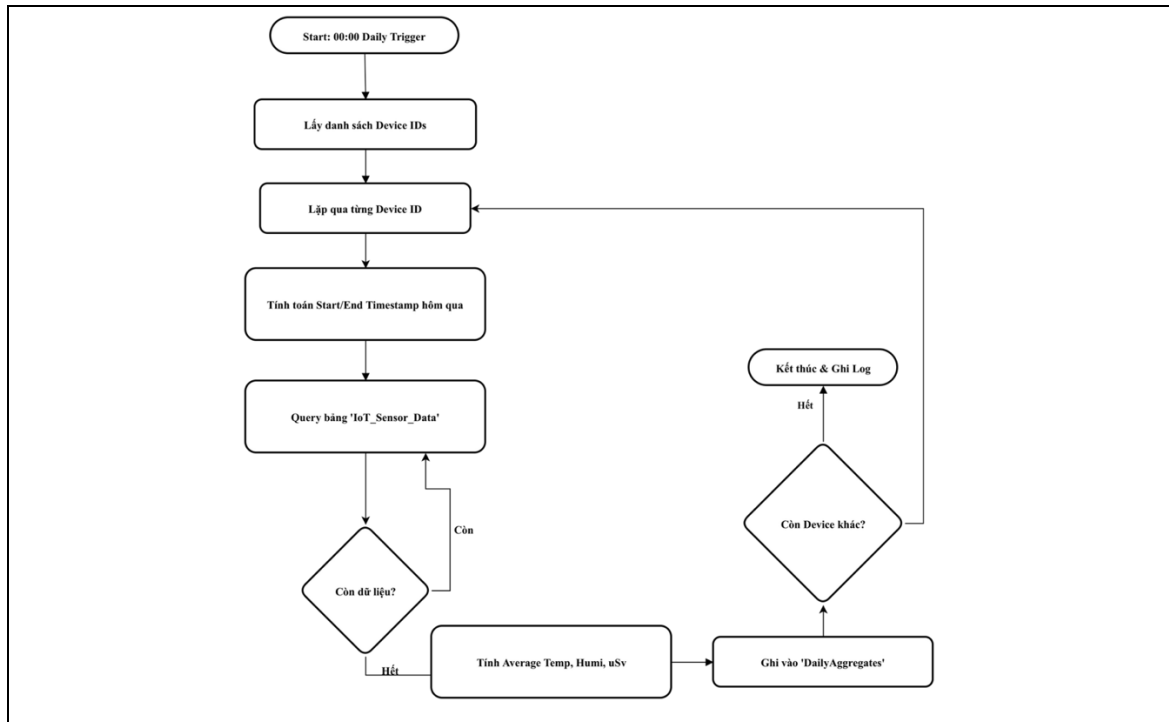
Trung tâm của quá trình tiếp nhận và phân luồng dữ liệu chính là hàm `lambda_process_incoming_data.py`. Hàm này được kích hoạt tự động theo thời gian thực mỗi khi AWS IoT Core nhận được bản tin MQTT từ thiết bị. Khi rule engine của IoT Core lắng nghe topic “tram/pub” sẽ tiến hành lọc bản tin và chuyển tiếp toàn bộ payload JSON đến hàm lambda để thực hiện xử lý dữ liệu theo cơ chế logic ETL - Extract-Transform-Load.

- Cơ chế Extract - trích xuất sẽ thực hiện bóc tách các trường dữ liệu được đóng gói trong JSON nhằm lấy các trường dữ liệu quan trọng và cần thiết.
- Cơ chế Transform - chuyển đổi sẽ là bước quan trọng nhằm chuyển đổi toàn bộ dữ liệu số thực float từ ngôn ngữ C++ của Arduino thường không tương thích trực tiếp với kiểu dữ liệu của DynamoDB sang kiểu decimal do hàm lambda ép buộc chuyển đổi để đảm bảo độ chính xác toán học, đồng thời chuyển đổi các biến đếm counts sang số nguyên init.

- Cơ chế Load - phân luồng lưu trữ sẽ giúp dữ liệu sau khi được chuyển đổi sang decimal sẽ ghi song song vào 3 bảng giá trị được định nghĩa trên DynamoDB:
 - Bảng IoT_Sensor_Data: dùng để lưu trữ vĩnh viễn toàn bộ thông tin để phục vụ cho quá trình đọc giá trị hoặc gọi vẽ biểu đồ lịch sử chi tiết.
 - Bảng DeviveStatus được sử dụng chỉ cập nhật duy nhất trường dữ liệu “lastSeen” nhằm để giao diện dashboard của website hoặc giao diện Admin có thể biết hệ thống trạm quan trắc đang ngoại tuyến hay trực tuyến. Bảng giá trị này chỉ nhận 01 trường dữ liệu nên bảng khá nhẹ nên việc gọi dữ liệu sẽ nhanh hơn cho với bảng dữ liệu thô IoT_Sensor_Data.
 - Bảng SystemLogs sẽ thực hiện việc chủ động phát hiện chỉ số phóng xạ vượt ngưỡng được thiết lập trong mã nguồn của các trạm nhằm đưa lên hệ thống bảng giá trị cảnh báo cấp độ tương ứng.

2.3.1.2 Tổng hợp dữ liệu định kỳ và tối ưu chi phí vận hành

Một hệ thống IoT hoạt động hoàn chỉnh thì mặt chi phí lưu trữ và độ trễ truy vấn chính là một vấn đề lớn có thể khiến cho hệ thống IoT dễ gặp lỗi timeout. Với tần suất gửi tin 7 giây/lần, bảng IoT_Sensor_Data có thể chứa hàng triệu bản ghi sau một năm, làm chậm đáng kể việc vẽ biểu đồ dài hạn. Nếu truy vấn trực tiếp vào bảng này để vẽ biểu đồ năm thì hệ thống sẽ phải quét hàng triệu dòng gây ra độ trễ lớn và tiêu tốn chi phí đọc RCU - Read Capacity Units. Để giải quyết vấn đề trên, hệ thống mới đã thực hiện triển khai hàm `lambda_aggregate_daily_data.py` nhằm giải quyết vấn đề trên với hoạt động như một tiến trình chạy nền và luôn luôn hiện diện trong hệ thống, chúng hoạt động như một bộ máy nén dữ liệu thông minh.



Hình 2.5. Giải thuật tổng hợp dữ liệu và phân luồng dữ liệu

Hình 2.5 là giải thuật về tổng hợp và phân luồng dữ liệu. Hàm lambda không chạy theo sự kiện thời gian thực mà được kích hoạt bởi Amazon EventBridge theo lịch trình cố định vào lúc 00:00:00 UTC +7 hằng ngày. Quy trình thực thi gồm 4 bước chính:

- **Bước 1: Xác định khung thời gian:** Tính toán chính xác thời điểm bắt đầu và kết thúc của ngày hôm trước dựa trên múi giờ Việt Nam.
- **Bước 2: Truy vấn phân trang:** Sử dụng vòng lặp while kết hợp với hàm “LastEvaluatedKey” để quét toàn bộ dữ liệu thô của từng thiết bị trong bảng giá trị IoT_Sensor_Data. Kỹ thuật này đảm bảo không bỏ sót bất kỳ dữ liệu nào ngay cả khi số lượng bản ghi vượt quá giới hạn 1MB của một lần gọi API DynamoDB.
- **Bước 3: Tính toán thống kê:** Thực hiện tính toán giá trị trung bình cộng cho nhiệt độ, độ ẩm và phóng xạ; đồng thời đếm tổng số điểm dữ liệu để đánh giá độ tin cậy của thông số thô.
- **Bước 4: Lưu trữ tóm tắt:** Kết quả cuối cùng chỉ gồm 1 dòng dữ liệu duy nhất cho mỗi trạm trong 1 ngày được ghi trực tiếp vào bảng DailyAggregates.

2.3.2 Lập trình API quản trị và điều khiển thiết bị

Trong hệ thống IoT hiện đại, khả năng giám sát trạng thái kết nối và điều khiển thiết bị từ xa là yêu cầu cần thiết. Để thực hiện điều này mà không cần lập các kết nối VPN phức

tập hay mở cổng kém bảo mật trên router tại thời điểm lắp đặt, hệ thống mới sử dụng cơ chế trung gian qua AWS IoT Core được điều phối bởi hàm lambda chuyên biệt. Phân hệ này bao gồm hai nhóm chức năng chính là giám sát trạng thái và điều khiển ngược chiều.

2.3.2.1 Cơ chế cầu nối điều khiển (HTTP to MQTT Bridge)

Hàm `lambda_device_control.py` hoạt động như một cầu nối giao thức nhằm chuyển đổi các yêu cầu HTTP RESTful từ giao diện Website thành các bản tin MQTT để giao tiếp với thiết bị phần cứng. Quy trình xử lý một lệnh điều khiển diễn ra qua 3 lớp bảo vệ xử lý:

- **Lớp xác thực và phân quyền (Authentication & Authorization):** Trước khi thực thi bất kỳ lệnh nào, hàm lambda thực hiện kiểm tra nghiệm ngặt danh tính người dùng thông qua Amazon Cognito. Hàm hỗ trợ `verify_admin` sẽ giải mã Token JWT trong tiêu đề Authorization, trích xuất thông tin người dùng và kiểm tra xem người dùng đó có thuộc nhóm admin hay không. Cơ chế này đảm bảo tuyệt đối không có truy cập trái phép nào có thể can thiệp vào hoạt động của trạm quan trắc.
- **Lớp đóng gói và định tuyến lệnh:** Sau khi xác thực thành công quyền Admin thì yêu cầu từ Website bao gồm các lệnh `stationId`, `command`, `payload` đều được đóng gói lại thành một bản tin JSON chuẩn. Hệ thống sử dụng cơ chế định tuyến động để xác định chính xác Topic MQTT đích cho từng trạm.
- **Lớp truyền tải thông tin dữ liệu:** Thư viện `boto3` được khai báo trong hàm `lambda` sẽ được sử dụng để publish bản tin xuống AWS IoT Core với cấp độ dịch vụ “QoS=1” nhằm đảm bảo lệnh chắc chắn được chuyển tới thiết bị ngay cả khi kết nối mạng chập chờn.

2.3.2.2 Cơ chế giám sát trạng thái thực

Việc giám sát bằng cách ping liên tục dữ liệu gây tốn nhiều băng thông dữ liệu, hệ thống áp dụng cơ chế nhịp tim thụ động thông qua hàm `lambda_get_device_status.py`. Nguyên lý hoạt động khi mỗi trạm gửi dữ liệu cảm biến thì bảng `DeviceStatus` trong DynamoDB sẽ tự động cập nhật trường `lastSeen` - thời điểm cuối nhìn thấy thiết bị nhằm giúp người dùng có thể xác định được hệ thống có thực hiện trực tuyến hay không. Khi người dùng mở trang Dashboard hàm `lambda` sẽ so sánh thời gian hiện tại “`current_time`” với “`lastSeen`” theo đúng cấu trúc sau:

- Thiết bị được cho là trực tuyến nếu khoảng cách thời gian nhỏ hơn ngưỡng “`OFFLINE_THRESHOLD_SECONDS`” được cài đặt là 30 giây.

- Ngược lại, thiết bị được cho là ngoại tuyến và hệ thống sẽ trả về chi tiết thời điểm mất kết nối để phục vụ việc tra soát lại hệ thống.

2.3.3 Triển khai AI Chatbot và hệ thống phân tích thông minh

Trong khi các hệ thống quan trắc truyền thống chỉ dừng lại ở việc hiển thị dữ liệu thô, hệ thống mới nâng cấp trải nghiệm người dùng bằng việc tích hợp trí tuệ nhân tạo - AI. Phân hệ này không hoạt động như một hợp chat thông thường mà đóng vai trò như một trợ lý vận hành ảo có khả năng hiểu ngữ cảnh hệ thống và truy xuất dữ liệu thời gian thực để đưa ra các tư vấn chính xác.

Trung tâm của phân hệ này là hàm `lambda_chay_ai.py` hoạt động dựa trên kỹ thuật RAG - Retrieval-Augmented Generation được tùy biến riêng cho hạ tầng Serverless.

2.3.3.1 Thiết kế bộ định tuyến thông minh với Prompt Engineering

Thay vì để AI trả lời tự do sẽ dễ dẫn đến trả lời sai, hệ thống áp dụng kỹ thuật gợi ý đầu ra có cấu trúc. Trong hàm `lambda_chat_ai.py` có một “System Prompt - Chỉ thị hệ thống” nghiêm ngặt được thiết lập để định hình hành vi của mô hình ngôn ngữ Google Gemini 2.5 Flash.

Hệ thống chỉ thị này sẽ yêu cầu AI không trả lời ngay câu hỏi của người dùng mà phải phân tích ý định và trả về một đối tượng JSON định nghĩa hành động cần thực hiện. Cách tiếp cận này sẽ biến ngôn ngữ tự nhiên của con người thành các lệnh máy tính chính xác nhờ cấu trúc (“SET_SENSOR_DATA”, “station_01”) giúp hệ thống ổn định và có kiểm soát.

2.3.3.2 Cơ chế gọi hàm chéo

Điểm khác biệt về mặt kiến trúc BE là khả năng tự động điều phối của Chatbot. Hàm `lambda_chat_ai.py` không trực tiếp truy vấn cơ sở dữ liệu vì chúng hoàn toàn vi phạm nguyên tắc đơn nhiệm - Single Responsibility. Thay vào đó, nó đóng vai trò là một chỉ huy có quyền gọi ngược lại các hàm lambda chức năng khác trong hệ thống nhằm để lấy dữ liệu thực tế và chính xác. Quy trình xử lý diễn ra theo mô hình khép kín:

- **Bước 1: Tiếp nhận:** Người dùng hỏi “Chỉ số phóng xạ trạm 3 có cao không?”
- **Bước 2: Phân tích:** Gemini API trả về JSON

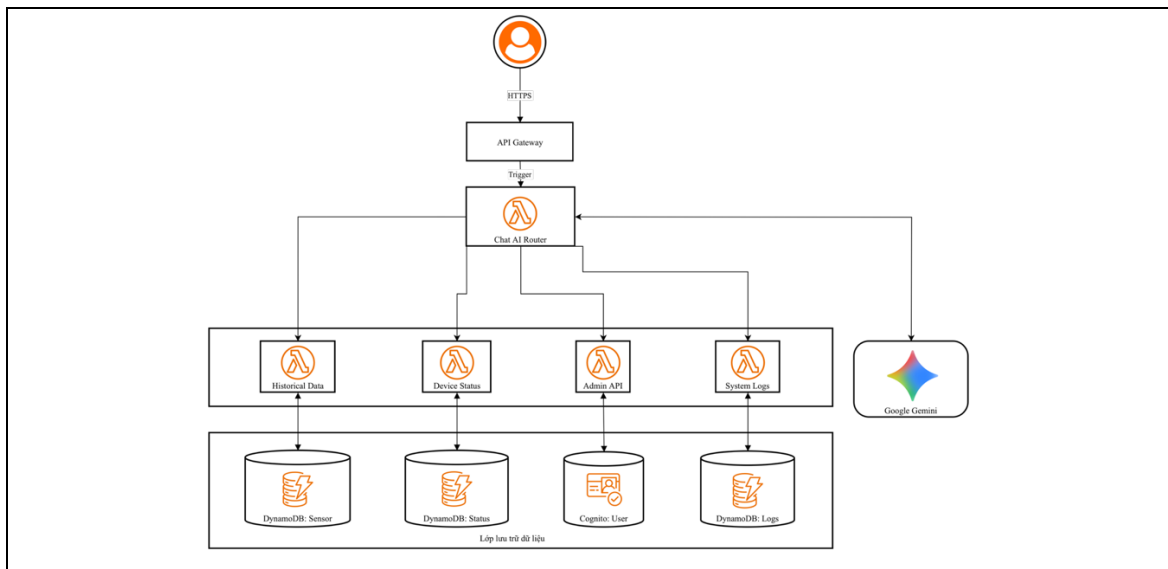
```

{{
    "action": "GET_SENSOR_DATA",
    "station": "station_03"
}}
```

- **Bước 3: Thực thi:** Hàm `lambda_chat_ai.py` sẽ sử dụng thư viện `boto3` để gọi ngược lại hàm `lambda_get_iot_data` hoặc hàm `lambda_get_historical_data`.
- **Bước 4: Tổng hợp:** Kết quả dữ liệu thô từ hàm con sẽ được AI sử dụng để tạo ra câu trả lời cuối cùng mang tính tư vấn.

2.3.3.3 Tính năng lưu trữ và quản lý tri thức

Không chỉ dừng lại ở việc hỏi đáp tức thời, hệ thống BE còn được trang bị khả năng lưu trữ các phiên phân tích giá trị để phục vụ công tác báo cáo về sau. Hai hàm `lambda` hỗ trợ đã được xây dựng là hàm `lambda_save_analysis.py` cho phép người dùng lưu lại các đoạn hội thoại hoặc kết quả phân tích bất thường vào bảng `DynamoDB Saved_Analyses`, mỗi bản ghi đều gán một ID duy nhất và nhãn thời gian thực cùng với hàm `lambda_get_saved_analyses.py` cung cấp API để trích xuất lại các phân tích đã lưu, hỗ trợ phân trang để xử lý lượng lớn bản ghi mà không gây quá tải.



Hình 2.6. Sơ đồ luồng dữ liệu của AI Chatbot

2.3.4 Xây dựng quy trình CI/CD và quản lý phiên bản tập trung

Trong quá trình phát triển hệ thống IoT hoàn chỉnh, việc viết và hoàn thành mã nguồn là điều cấp thiết nhưng khâu triển khai và bảo trì hệ thống cũng là một vấn đề. Các phương pháp truyền thống như upload thủ công file firmware lên máy chủ FTP hay sao chép file giao diện Website vào hosting thường tiềm ẩn nhiều rủi ro sai sót phiên bản và tốn kém thời gian. Để giải quyết triệt để áp dụng quy trình CI/CD hiện đại kèm theo sử dụng kho lưu trữ GitHub và AWS Amplify làm nền tảng vận hành tự động.

2.3.4.1 Quản lý đăng nhập và kiểm tra quyền quản trị hệ thống

Hệ thống không lưu trữ mật khẩu người dùng trong cơ sở dữ liệu thông thường vì khả năng bảo mật kém và rủi ro bị tấn công rất cao mà chúng đã được tích hợp sâu với Amazon Cognito. Các hàm lambda đóng vai trò là lớp proxy xử lý logic nghiệp vụ trước khi giao tiếp chính thức Cognito. Quy trình xác thực được thực hiện chặt chẽ và liên kết giữa 2 hàm `lambda_register.py` và hàm `lambda_login.py`. Chúng xử lý việc đăng ký và đăng nhập của toàn bộ hệ thống. Khi đăng nhập thành công, hệ thống trả về bộ 3 tokens chuẩn theo giao thức OAuth2 bao gồm các trường thông tin như `IdToken`, `AccessToken` và `RefreshToken`. Toàn bộ API khác trong hệ thống như điều khiển thiết bị hai chiều đều yêu cầu “Bearer Token” trong header.

Đối với các tài khoản là quản trị sẽ phải thực hiện phân quyền quản trị. Hàm `lambda_admin_api.py` cung cấp các tính năng đặc biệt cho nhóm admin như có thể xem toàn bộ danh sách người dùng “`LIST_USERS`”, phê duyệt yêu cầu nâng cấp gói người dùng “`PROCESS_UPGRADE_REQUEST`) hoặc vô hiệu hoá tài khoản vi phạm “`TOGGLE_USER_STATUS`). Hàm `lambda_request_upgrade.py` cho phép người dùng thường xuyên gửi yêu cầu nâng cấp tài khoản, tạo ra một quy trình phê duyệt tự động hoá trên nền tảng Serverless.

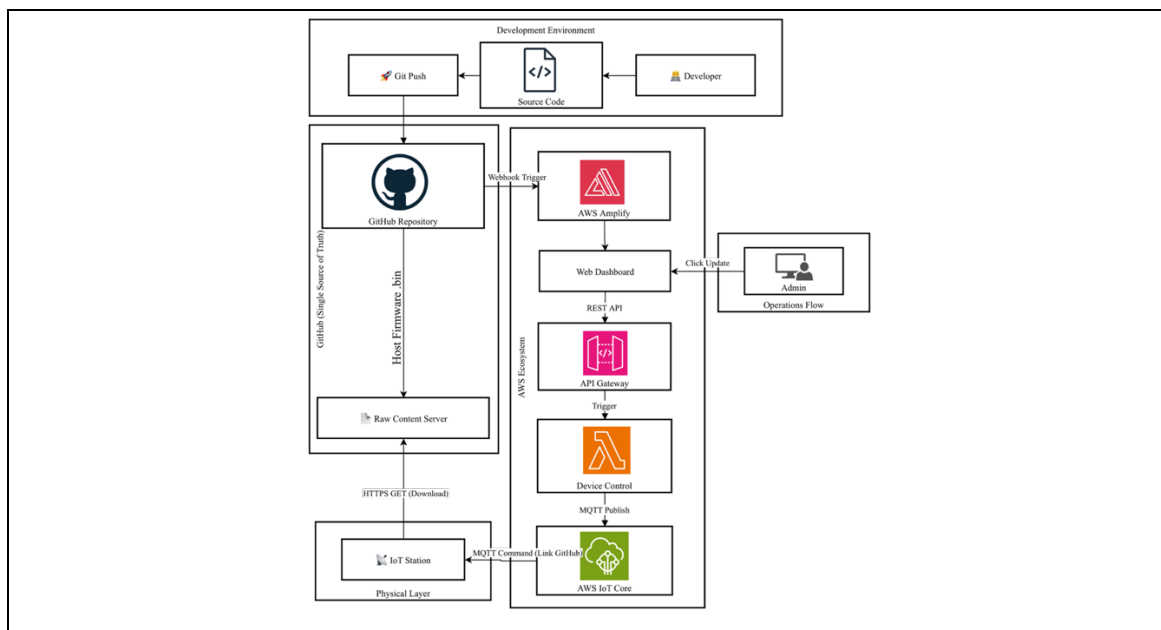
2.3.4.2 Quản lý vòng đời firmware và cơ chế OTA qua CDN

Thay vì xây dựng một máy chủ tải file thủ công riêng lẻ như S3 Bucket phải đòi hỏi cấu hình phân quyền phức tạp, hệ thống tận dụng hạ tầng CDN - Content Delivery Network toàn cầu của GitHub để phân phối firmware. Mọi thay đổi trong mã nguồn nhúng C++ đều được quản lý chặt chẽ bằng Git. Khi một phiên bản firmware mới được kiểm thử thành công và biên dịch ra tệp nhị phân “.bin” nó sẽ được commit lên nhánh chính của GitHub. Lúc này GitHub sẽ tự động cung cấp một đường dẫn truy cập trực tiếp tới tệp tin đã tải lên. Đường dẫn này đóng vai trò là nguồn tải xuống tin cậy, tốc độ cao và ổn định cho các thiết bị IoT.

Về cơ chế kích hoạt cập nhật OTA, để kích hoạt quá trình cập nhật trên thiết bị, hệ thống sử dụng hàm `lambda_device_control.py` như một pyaload đặc biệt. Thay vì gửi lệnh điều khiển phần cứng như bật/tắt nguồn thì admin gửi lệnh `UPDATE_FIRMWARE` kèm theo đường dẫn URL từ GitHub.

2.3.4.3 Tự động hoá triển khai Web với AWS Amplify

Đối với giao diện Dashboard quản trị, hệ thống mới loại bỏ hoàn toàn quy trình như hệ thống tiền nhiệm “Build thủ công → Upload thủ công”.



Hình 2.7. Sơ đồ luồng CI/CD và OTA tổng hợp

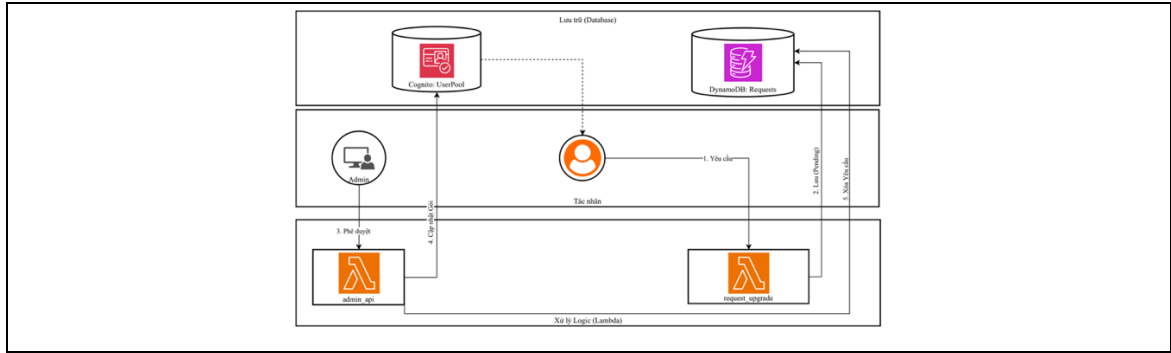
Thay vào đó, dịch vụ AWS Amplify Console được kết nối trực tiếp với kho chứa mã nguồn GitHub. Cơ chế webhook và build pipeline của AWS Amplify sẽ lắng nghe các sự kiện thay đổi mã nguồn từ GitHub. Ngay khi lập trình viên thực hiện lệnh “git push” lên nhánh chính main thì một chuỗi hành động sẽ được kích hoạt tự động:

- Provision: AWS Amplify cấp phát một container môi trường sạch.
- Build: Tải mã nguồn, cài đặt thư viện “npm install” và biên dịch ứng dụng FE.
- Deploy: Đẩy các tệp tĩnh gồm HTML/CSS/JS sang mạng lưới phân phối nội dung CDN và cập nhật phiên bản web mới nhất cho người dùng toàn cầu chỉ trong vài phút.

Việc quản lý các biến môi trường bảo mật của các thông số như API_ENDPOINT, COGNITO_USER_POOL_ID không được lưu cứng trong mã nguồn GitHub để tránh lộ thông tin ra ngoài. Thay vào đó, chúng được cấu hình trong phần Environment Variables của AWS Amplify và được tiêm vào ứng dụng trong quá trình build. Điều này đảm bảo tuân thủ các tiêu chuẩn bảo mật DevSecOps.

2.3.5 Tích hợp quản lý phân quyền và lưu trữ cá nhân

Hệ thống tích hợp thêm các hàm lambda hỗ trợ giúp quản lý định danh và dữ liệu riêng biệt nhằm đảm bảo tính bảo mật của hệ thống và khả năng cá nhân hoá.

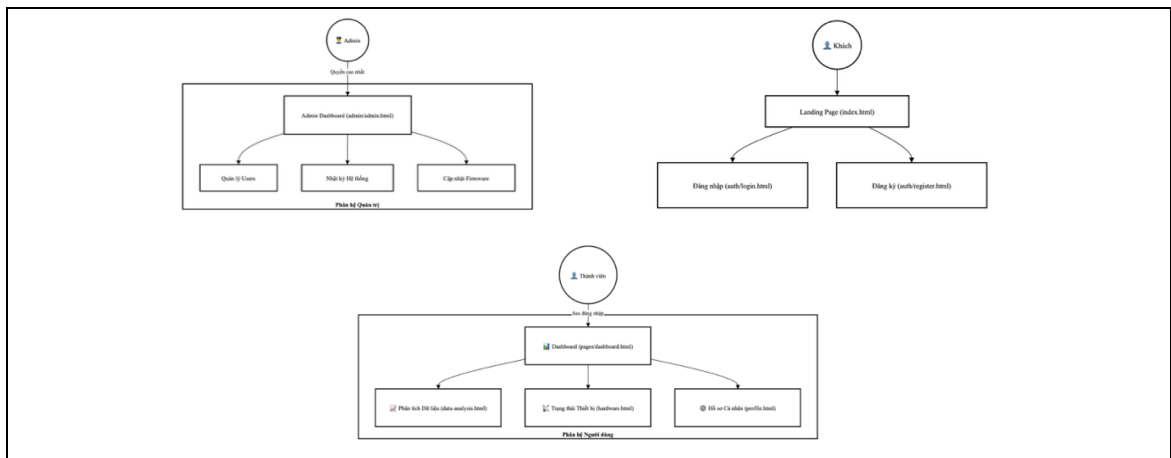


Hình 2.8. Sơ đồ quản lý và phê duyệt người dùng

Hình 2.8 trình bày sơ đồ hệ thống sử dụng kết hợp hàm lambda `admin_api.py` và Amazon Cognito để phân chia quyền hạn rõ ràng giữa Admin và User. Admin có quyền truy cập vào toàn bộ danh sách người dùng và can thiệp vào cấu hình hệ thống, trong khi người dùng thường chỉ được phép xem dữ liệu trực tuyến. Hàm `lambda_request_upgrade.py` xử lý logic nâng cấp quyền hạn một cách tự động hoá thông qua cơ chế hàng đợi trên DynamoDB. Ngoài ra, hệ thống còn cung cấp thêm khả năng lưu trữ các kết quả phân tích quan trọng thông qua hàm `lambda_save_analysis.py`. Tính năng này cho phép người dùng lưu lại các đoạn hội thoại với AI hoặc các báo cáo về cơ sở dữ liệu giúp việc truy xuất lại lịch sử trở nên dễ dàng và bảo mật hơn.

2.4 Thiết kế và cấu trúc hệ thống giao diện Web

Giao diện website được thiết kế theo kiến trúc MPA - Multi-page Applocation nhưng lại được tối ưu hoá trải nghiệm theo hướng mượt mà như kiến trúc CPA - Single Page Application. Hình 2.9 minh họa 3 luồng truy cập cho Admin, khách và thành viên. Các tài nguyên tĩnh bao gồm CSS3, JS được dùng chung trong khi nội dung HTML được chia thành các trang riêng biệt phục vụ cho từng chức năng riêng. Hệ thống được chia thành 3 luồng truy cập dựa trên vai trò của người dùng.



Hình 2.9. Sơ đồ cấu trúc Website theo quyền Admin, Khách và thành viên

CHƯƠNG 3 – KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ HIỆU NĂNG

Trong chương 3, chúng tôi thực hiện đánh giá hoạt động của hệ quan trắc

3.1 Đánh giá hoạt động của hệ quan trắc phóng xạ thời gian thực

3.1.1 Kiểm tra dữ liệu và kết nối

Sau khi xây dựng hệ thống, chúng tôi trước hết kết thực hiện kiểm tra truyền dữ liệu của hệ quan trắc. Cụ thể, hệ thống được kết nối với máy tính thông qua cổng USB dựa vào chuẩn Serial Monitor của phần mềm Arduino IDE.

Cụ thể là kiểm tra trạng thái kết nối WiFi/MQTT, kiểm tra kết nối với server AWS IOT, cũng như kiểm dữ liệu được truyền ra.

Hình 3.1 là hình ảnh hiển thị kết nối và truyền dữ liệu ra thông qua truy xuất từ Serial Monitor. Kết quả thể hệ thống đã kết nối thành công WiFi/MQTT, AWS và dữ liệu được truy xuất.

```
rst:Connecting WiFi...
WiFi Connected
WiFi connected (EEPROM check)
Setting time using SNTP.NTPConnect..NTPConnect..NTPConnect..NTPConnect..NTPConnect.done!
Current time: Sat Jan 10 09:58:45 2026
Connecting to AWS IOT
AWS IoT Connected!
[LOOP] Time: 00:00:10 | Counts: 14 | CPS: 1.79 | uSv/h: 0.711 | Temp: 28.10
[LOOP] Time: 09:58:47 | Counts: 14 | CPS: 0.00 | uSv/h: 0.000 | Temp: 28.10
[LOOP] Time: 09:58:48 | Counts: 14 | CPS: 0.00 | uSv/h: 0.000 | Temp: 28.10
[LOOP] Time: 09:58:49 | Counts: 14 | CPS: 0.00 | uSv/h: 0.000 | Temp: 28.10
[LOOP] Time: 09:58:51 | Counts: 14 | CPS: 0.00 | uSv/h: 0.000 | Temp: 28.10
[LOOP] Time: 09:58:52 | Counts: 14 | CPS: 0.00 | uSv/h: 0.000 | Temp: 28.10
[LOOP] Time: 09:58:53 | Counts: 23 | CPS: 8.79 | uSv/h: 3.492 | Temp: 28.00
```

Hình 3.1. Kiểm tra dữ liệu ghi nhận của hệ đo và kiểm tra kết nối WiFi/MQTT, AWS

3.1.2 Kiểm tra hệ thống tự kết internet

Để hệ thống có thể vận hành lâu dài tại các vị trí không có người giám sát, firmware được tích hợp cơ chế Watchdog giúp thiết bị tự động khởi động lại khi phát hiện sự cố treo mạng hoặc mất kết nối kéo dài. Khi mạng WiFi hoặc máy chủ AWS gặp sự cố gián đoạn, thiết bị sẽ không thể thiết lập kết nối MQTT và sẽ bị kẹt lại ở bước “.connectAWS” và in ra các dấu chấm chờ đợi.

Với hệ thống được thiết lập thời gian 70 giây, nếu sự kết nối WiFi/MQTT, AWS bị gián đoạn, hệ thống sẽ tự kích hoạt lệnh khởi động ESP.restart(). Hình 3.2 được thiết lập để kiểm tra hệ thống. Dựa vào truy nhập hệ thống, cho thấy mã lỗi khởi động là “rst:0xc (SW_CPU_RESET)” được kích hoạt, và thực hiện “reset” để kết nối Wifi, cũng như kết nối AWS.

```

Current time: Sat Jan 10 09:18:19 2026
Connecting to AWS IOT
.connectAWS.ets Jul 29 2019 12:21:46

rst:0xc (SW_CPU_RESET) boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:4744
load:0x40078000,len:15712
load:0x40080400,len:3164
entry 0x4008059c
Connecting WiFi...
WiFi Connected
WiFi connected (EEPROM check)
Setting time using SNTPdone!
Current time: Sat Jan 10 09:20:26 2026
Connecting to AWS IOT

```

Hình 3.2. Ghi nhận hệ thống tự kết nối khi có sự cố mất Wifi.

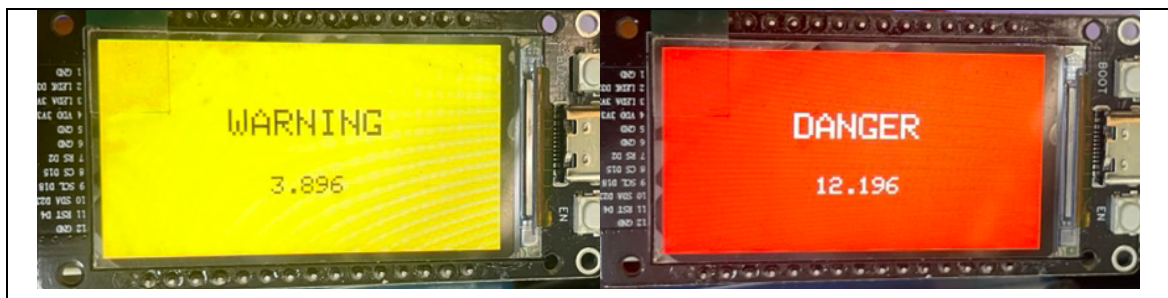
3.1.3 Kiểm tra hiển thị và cảnh báo trên LCD

Sau khi toàn bộ quá trình kết nối WiFi/MQTT được kết nối thành công sẽ hiển thị màn hình với các thông số đã được định nghĩa trong code nhúng. Tại hình 3.4 hiển thị dữ liệu ghi nhận theo thời gian thực đã hoạt động với các thông số đo từ cảm biến nhiệt độ - độ ẩm, thông số đo từ đầu dò GM như số đếm tích lũy, CPS,... và thời gian thực tế tại trạm đo.



Hình 3.3. Hiển thị dữ liệu ghi nhận theo thời gian thực

Đối với hiển thị cảnh báo, màn hình báo hiệu cảnh báo vượt ngưỡng được thể hiện dưới dạng hiển thị màu. Cụ thể, màu cam vượt ngưỡng cảnh báo; màu đỏ là nguy hiểm, như được thể hiện ở hình 3.4.



Hình 3.4. Hình thị cảnh báo quá ngưỡng tại LCD

3.1.4 Kiểm tra dữ liệu được gửi lên AWS

Chúng tôi kiểm tra dữ liệu truyền lên AWS. Hình 3.5 là kết quả dữ liệu ghi nhận tại AWS do dữ liệu từ hệ đo truyền lên. Dữ liệu ghi nhận tại AWS dạng dữ liệu JSON nhận được trên AWS IoT Core. Dữ liệu ghi nhận gồm có thời gian ghi nhận tại hệ đo; tên trạm truyền lên; dữ liệu đo từ cảm biến nhiệt độ/độ ẩm và dữ liệu ghi nhận từ đầu dò GM. Dữ liệu này được cập nhật với thời gian được thiết lập 7-8 giây. Hình 3.5 là dữ liệu của hai sự kiện ghi nhận.

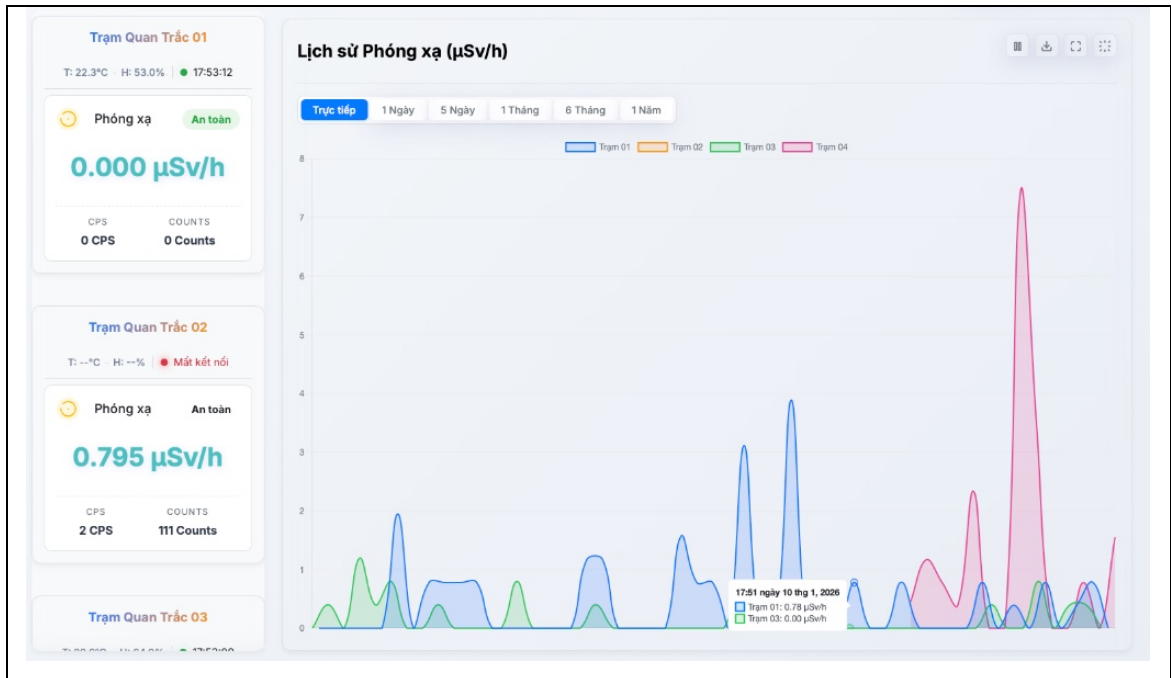
```
{
  "LocalTime": "Sun Jan 11 14:38:52 2026",
  "deviceId": "ESP8266_Station_4",
  "cps": 0,
  "uSv": 0,
  "timestamp": 1768117132,
  "stationName": "Tram4",
  "humidity": 45,
  "counts": 0,
  "temperature": 29
},
{
  "LocalTime": "Sun Jan 11 14:39:00 2026",
  "deviceId": "ESP8266_Station_4",
  "cps": 1.970443,
  "uSv": 0.782958,
  "timestamp": 1768117140,
  "stationName": "Tram4",
  "humidity": 45,
  "counts": 0,
  "temperature": 29
},
```

Hình 3.5. Hình ảnh dữ liệu thực tế AWS nhận được từ hệ đo

3.2 Triển khai giao diện website cho hệ đo

3.2.1 Web hiển thị dữ liệu quan trắc thời gian thực

Hình 3.6 web hiển thị dữ liệu quan trắc theo thời gian thực. Giao diện được thiết kế theo theo 2 vùng hiển thị. Cột bên trái hiển thị thẻ thông tin của từng trạm với các thông số tức thời kèm theo trạng thái hoạt động và khu vực bên phải hiển thị biểu đồ dữ liệu đo phóng xạ theo thời gian thực. Ở hiển thị dạng đồ thị cho phép, hiển thị số liệu tức thời (trong khoảng thời gian ngắn), và đặc biệt, người dùng có thể xem dữ liệu lịch sử theo các mốc 1 ngày, 5 ngày, 1 tháng, 6 tháng hay 1 năm. Hơn nữa, người dùng có thể chọn/tắt trạm dữ liệu để hiển thị.

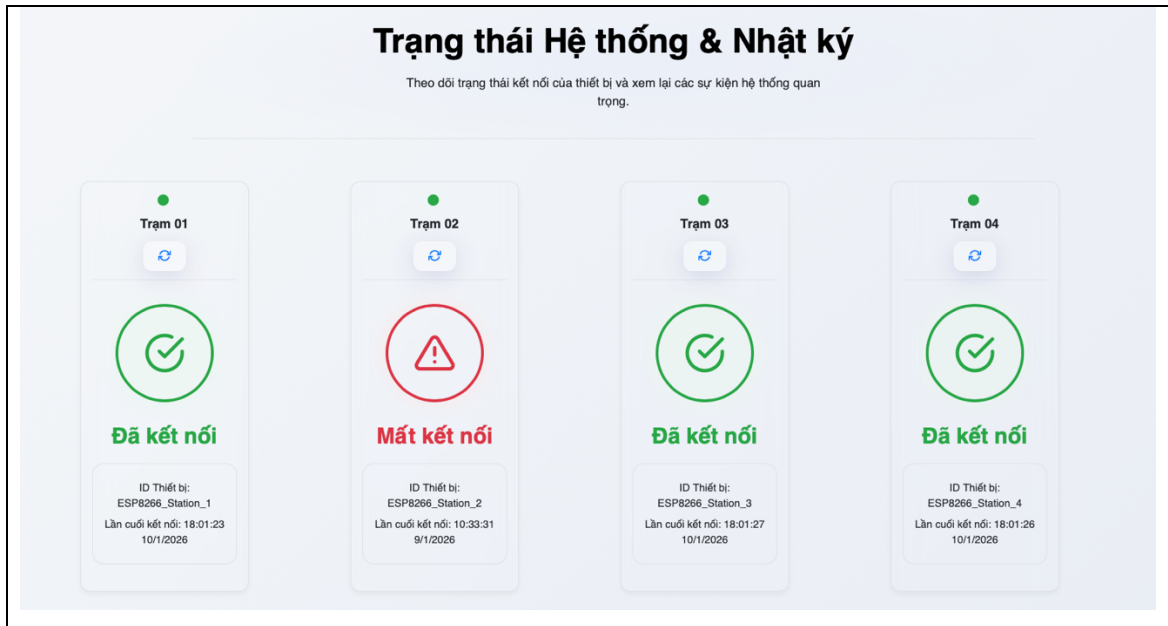


Hình 3.6. Giao diện Web hiển thị dữ liệu đo cho 4 trạm.

3.2.2 Trang trạng thái và nhật ký hệ thống

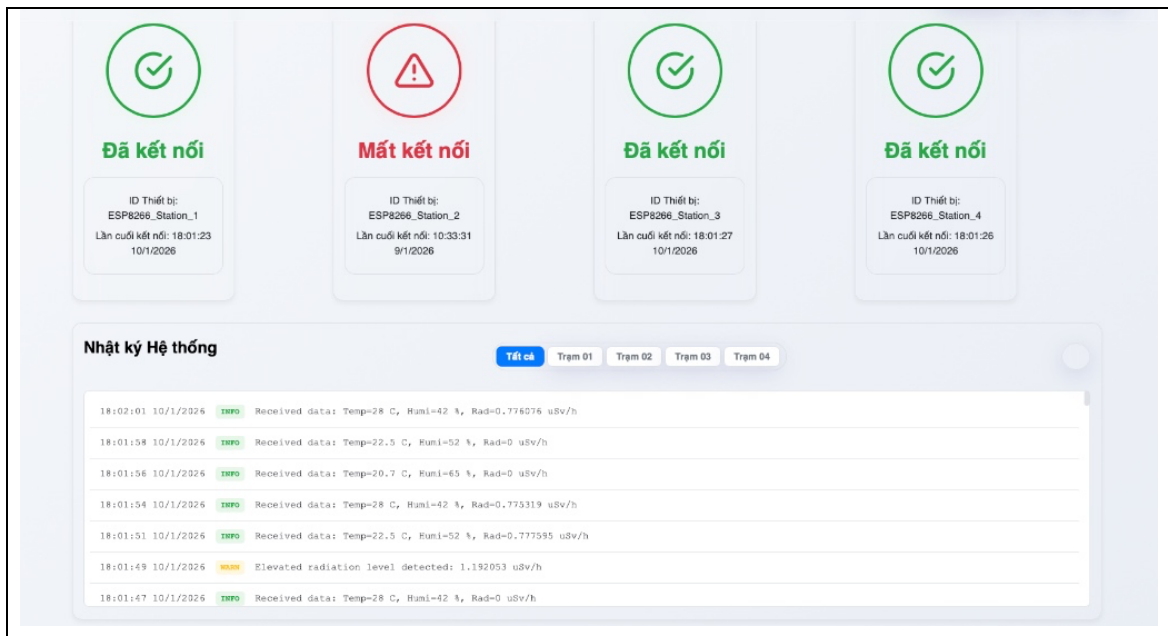
Bên cạnh giao diện WEB hiển thị dữ liệu đo cho 4 trạm trực quan hoá dữ liệu, hệ thống Web bổ sung trang trạng thái - status.html là trang Web hỗ trợ người dùng giám sát tình trạng của hệ thống quan trắc và truy xuất dữ liệu lịch sử hoạt động cũng như các thông số dữ liệu thô thông qua bảng nhật ký - system logs.

Hình 3.7 là hình ảnh thực tế về trang Web trạng thái hệ thống, giao diện sử dụng các thẻ thông tin UI để hiển thị trạng thái của từng trạm trong mạng lưới. Hệ thống liên tục kiểm tra thời gian của dữ liệu cuối cùng nhận được từ trạm đo, nếu thời gian này nằm trong ngưỡng cho phép thì sẽ hiển thị trạng thái “Đã kết nối” màu xanh lá và vượt qua ngưỡng thời gian thì sẽ hiển thị trạng thái “Mất kết nối” màu đỏ.



Hình 3.7. Hình ảnh thực tế về trạng thái hệ thống. Màu xanh là kết nối thành công, màu đỏ là mất kết nối.

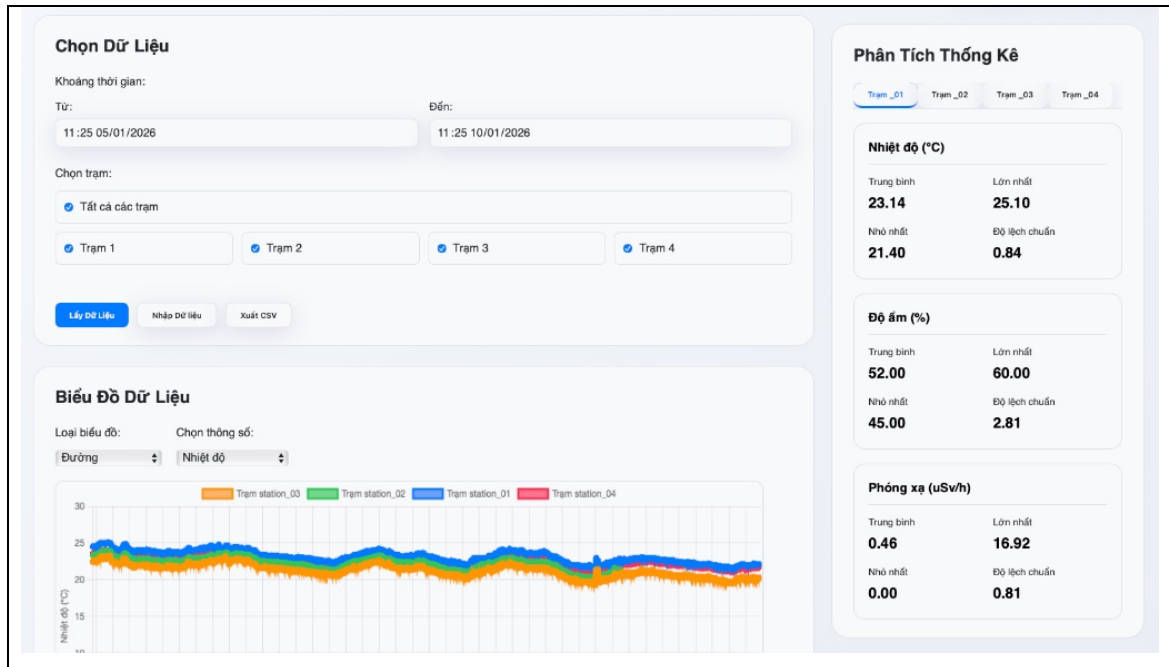
Ngoài ra, hình 3.8 nhật ký hệ thống cũng cung cấp về dữ liệu thô trực tiếp trên giao diện Web thay vì dữ liệu JSON. Mỗi dòng nhật ký bao gồm các thông tin về thời gian ghi nhận, mức độ, và nội dung kèm theo. Người dùng có thể lọc nhật ký theo từng trạm để có thể xem chi tiết và dễ dàng theo dõi.



Hình 3.8. Nhật ký hệ thống ghi nhận cảnh báo

3.2.3 Trang phân tích dữ liệu

Hình 3.9 là giao diện thiết lập tham số truy vấn dữ liệu của trang Web là trung tâm trong việc chuyển đổi dữ liệu thô thu thập từ hệ đo thành các thông tin có giá trị phục vụ cho nhiều công tác khác nhau.



Hình 3.9. Giao diện thiết lập tham số truy vấn dữ liệu

Giao diện truy vấn dữ liệu của hình 3.9 được thiết kế với các bộ lọc linh hoạt cho phép người dùng tùy biến phạm vi phân tích theo thời gian thực hoặc khoảng thời gian lịch sử cụ thể. Hệ thống hỗ trợ trích xuất dữ liệu đồng thời từ nhiều trạm để so sánh tương quan hoặc tách riêng từng trạm để đánh giá chi tiết. Các dữ liệu này được trực quan hoá thông qua hệ thống biểu đồ động, tự động điều chỉnh thang đo phù hợp với từng loại tham số môi trường như nhiệt độ, độ ẩm hay phóng xạ.

Kết quả trên biểu đồ cho thấy sự ổn định và độ chi tiết cao của dữ liệu thu thập được. Cụ thể tại hình 3.10a biểu đồ đường biểu diễn nhiệt độ của 4 trạm cho thấy sự biến thiên đồng nhất trong khoảng 20°C - 50°C phản ánh chính xác điều kiện môi trường thực tế. Đối với dữ liệu phóng xạ, biểu đồ phân tán tại hình 3.10b thể hiện rõ mật độ điểm dữ liệu dày đặc chứng minh thiết bị hoạt động liên tục với tần suất gửi ổn định.



Hình 3.10. Biểu đồ phân tích xu hướng nhiệt độ và mật độ phóng xạ

Bên cạnh khả năng hiển thị đồ họa, hệ thống tự động tính toán các chỉ số thống kê mô tả như giá trị trung bình, giá trị cực đại, cực tiểu, độ lệch chuẩn. Hình 3.11 là bảng phân tích thống kê chi tiết của 4 trạm khi được truy xuất dữ liệu, khi truy xuất hệ thống sẽ tính toán và xuất ra các kết quả thống kê. Ngoài ra, hình 3.12 là bảng giá trị .csv khi trích xuất dữ liệu từ trang phân tích hỗ trợ cho việc lưu trữ và xử lý dữ liệu trên các phần mềm chuyên dụng khác.

Phân Tích Thống Kê			
Trạm_01	Trạm_02	Trạm_03	Trạm_04
Nhiệt độ (°C)			
Trung bình	Lớn nhất		
23.14	25.10		
Nhỏ nhất	Độ lệch chuẩn		
21.40	0.84		
Độ ẩm (%)			
Trung bình	Lớn nhất		
52.00	60.00		
Nhỏ nhất	Độ lệch chuẩn		
45.00	2.81		
Phóng xạ (uSv/h)			
Trung bình	Lớn nhất		
0.46	16.92		
Nhỏ nhất	Độ lệch chuẩn		
0.00	0.81		

Phân Tích Thống Kê			
Trạm_01	Trạm_02	Trạm_03	Trạm_04
Nhiệt độ (°C)			
Trung bình	Lớn nhất		
21.25	23.30		
Nhỏ nhất	Độ lệch chuẩn		
19.30	0.85		
Độ ẩm (%)			
Trung bình	Lớn nhất		
65.29	71.00		
Nhỏ nhất	Độ lệch chuẩn		
61.00	1.62		
Phóng xạ (uSv/h)			
Trung bình	Lớn nhất		
0.14	2.78		
Nhỏ nhất	Độ lệch chuẩn		
0.00	0.25		

Phân Tích Thống Kê			
Trạm_01	Trạm_02	Trạm_03	Trạm_04
Nhiệt độ (°C)			
Trung bình	Lớn nhất		
22.51	24.10		
Nhỏ nhất	Độ lệch chuẩn		
20.90	0.75		
Độ ẩm (%)			
Trung bình	Lớn nhất		
60.57	70.00		
Nhỏ nhất	Độ lệch chuẩn		
54.00	3.07		
Phóng xạ (uSv/h)			
Trung bình	Lớn nhất		
0.42	12.43		
Nhỏ nhất	Độ lệch chuẩn		
0.00	0.82		

Phân Tích Thống Kê			
Trạm_01	Trạm_02	Trạm_03	Trạm_04
Nhiệt độ (°C)			
Trung bình	Lớn nhất		
22.29	23.90		
Nhỏ nhất	Độ lệch chuẩn		
20.10	0.83		
Độ ẩm (%)			
Trung bình	Lớn nhất		
60.36	73.00		
Nhỏ nhất	Độ lệch chuẩn		
53.00	3.76		
Phóng xạ (uSv/h)			
Trung bình	Lớn nhất		
0.19	3.18		
Nhỏ nhất	Độ lệch chuẩn		
0.00	0.30		

Hình 3.11. Bảng phân tích thống kê chi tiết của 4 trạm

Timestamp	Station	Temperature	Humidity	Radiation
2026-01-05 04:37:02	station_04	23.60	61.00	0.000
2026-01-05 04:37:03	station_03	22.40	64.00	0.000
2026-01-05 04:37:05	station_01	24.50	51.00	1.952
2026-01-05 04:37:06	station_02	23.30	59.00	0.000
2026-01-05 04:37:09	station_04	23.50	61.00	1.569
2026-01-05 04:37:10	station_03	22.40	64.00	0.000
2026-01-05 04:37:12	station_01	24.40	51.00	0.000
2026-01-05 04:37:13	station_02	23.30	59.00	0.000
2026-01-05 04:37:16	station_04	23.50	61.00	0.000
2026-01-05 04:37:17	station_03	22.40	64.00	0.000
2026-01-05 04:37:19	station_01	24.50	51.00	0.000
2026-01-05 04:37:20	station_02	23.30	59.00	0.000
2026-01-05 04:37:23	station_04	23.50	61.00	0.776
2026-01-05 04:37:24	station_03	22.40	64.00	0.000
2026-01-05 04:37:26	station_01	24.40	51.00	0.000
2026-01-05 04:37:28	station_02	23.30	59.00	0.000
2026-01-05 04:37:30	station_04	23.50	61.00	0.000
2026-01-05 04:37:31	station_03	22.40	64.00	0.000
2026-01-05 04:37:33	station_01	24.40	51.00	0.784
2026-01-05 04:37:35	station_02	23.30	59.00	0.000

Hình 3.12. Bảng giá trị csv khi trích xuất dữ liệu từ trang phân tích

3.3 Triển khai giao diện website hỗ trợ cho quản lý

3.3.1 Trang giao diện xác thực và bảo mật

Để đảm bảo an toàn cho hệ thống dữ liệu quan trọng, quyền truy cập vào các chức năng quản trị được kiểm soát chặt chẽ thông qua cơ chế đăng nhập bảo mật.

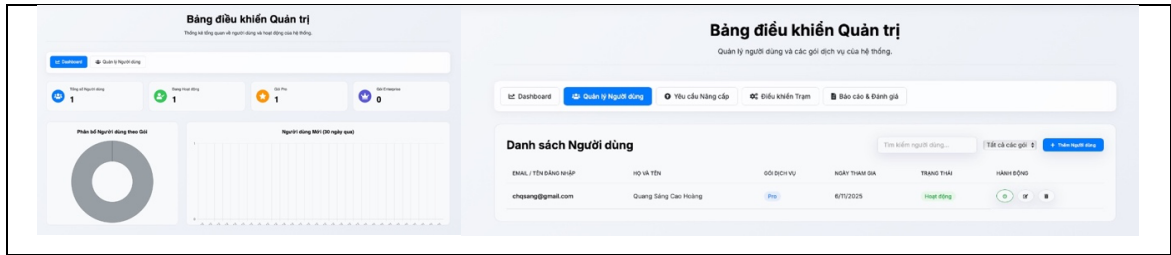
Hình 3.13. Giao diện đăng nhập của hệ thống

Hình 3.13 là giao diện đăng nhập của hệ thống được thiết kế tối giản, tập trung vào trải nghiệm người dùng, hệ thống yêu cầu thông tin email và mật khẩu đã được cấp phép. Về cơ chế bảo vệ, mật khẩu được mã hoá một chiều trước khi lưu trữ và đồng thời sử dụng token JWT để duy trì trạng thái đăng nhập, tự động hết hạn sau một khoảng thời gian không hoạt động để ngăn chặn truy cập trái phép. Khi đăng nhập hệ thống, người dùng nhập sai thông tin sẽ thông báo lỗi tức thời và khi đăng nhập thành công, người dùng sẽ được chuyển hướng vào trang chủ của trang web.

3.3.2 Trang quản trị hệ thống và các chức năng điều khiển

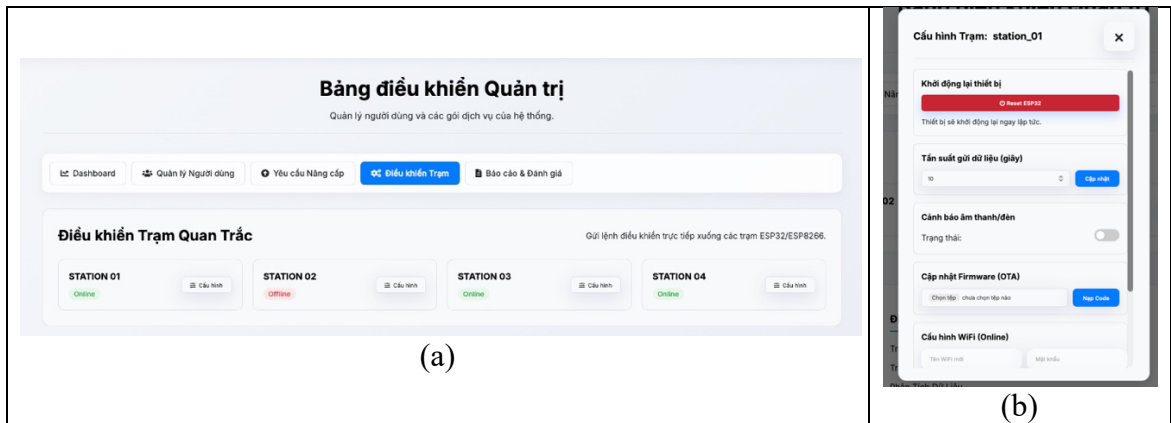
Hình 3.14 là giao diện tổng quan và quản lý người dùng cung cấp tổng quát về quy mô hệ thống thông qua các biểu đồ thống kê người dùng và phân loại dịch vụ. Chức năng quản lý người dùng cho phép quản trị viên thực hiện các thao tác phân quyền theo dõi trạng

thái hoạt động can thiệp khoá/mở khoá tài khoản khi cần thiết, đảm bảo tính bảo mật và kiểm soát chặt chẽ quyền truy cập vào dữ liệu quan trắc.



Hình 3.14. Giao diện tổng quan và quản lý người dùng

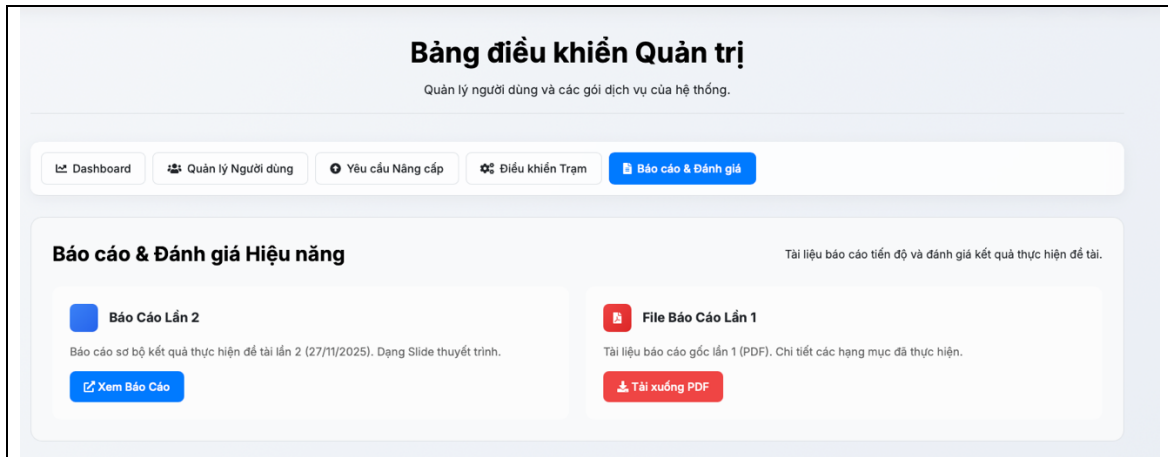
Điểm nổi bật về mặt kỹ thuật của phân hệ quản trị là khả năng giao tiếp hai chiều giữa trang Web và hệ đo.



Hình 3.15. Giao diện điều khiển trạm và cấu hình chi tiết

Tại giao diện điều khiển trạm hình 3.15a, trạng thái kết nối của toàn mạng lưới được áp dụng cập nhật đồng bộ với trang Web hiển thị dữ liệu của người dùng, thể hiện chính xác trạng thái trực tuyến của các trạm hoạt động và cảnh báo ngoại tuyến. Đặc biệt thông qua bảng điều khiển chi tiết hình 3.15b người quản trị có thể gửi các lệnh điều khiển trực tiếp xuống các trạm quan trắc thông qua giao thức MQTT. Các tính năng điều khiển bao gồm: Khởi động lại thiết bị để xử lý sự cố treo phần mềm từ xa; Điều chỉnh tần suất gửi dữ liệu giúp tối ưu năng lượng và đặc biệt là tính năng cập nhật firmware OTA và cấu hình mạng WiFi cho phép bảo trì và nâng cấp hệ thống mà không cần tiếp cận thiết bị tại hiện trường.

Ngoài ra, hình 3.16 là kho lưu trữ báo cáo và tài liệu tự động tổng hợp các báo cáo hiệu năng định kỳ và cho phép tải xuống. Tính năng này hỗ trợ cho công tác lưu trữ hồ sơ, theo dõi tiến độ và đánh giá hiệu quả vận hành của hệ thống theo thời gian.



Hình 3.16. Kho lưu trữ báo cáo và tài liệu

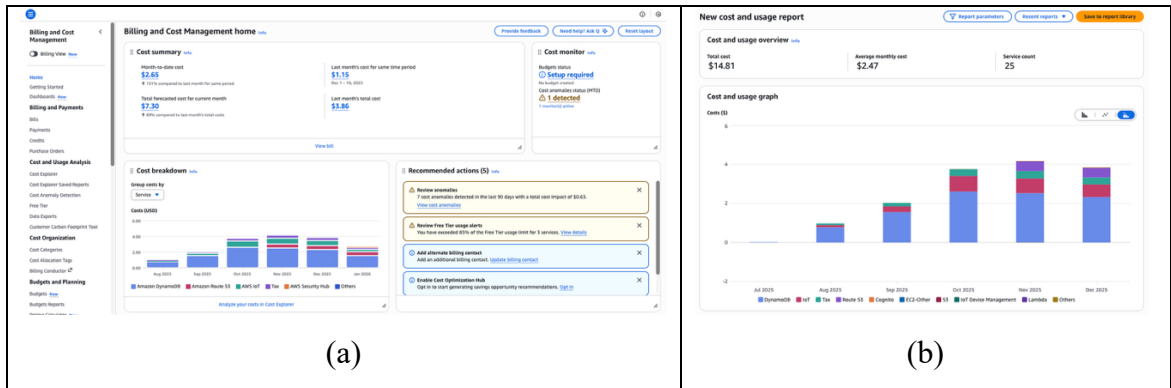
3.4 Đánh giá hiệu quả kinh tế và tối ưu chi phí vận hành

Bên cạnh các yếu tố kỹ thuật, tính khả thi của đề tài khi triển khai thực tế còn được đánh giá dựa trên chi phí vận hành. Công cụ AWS Cost Explorer [8] để theo dõi và phân tích chi phí hạ tầng đám mây trong giai đoạn thực hành đề tài từ tháng 11/2025 đến tháng 01/2026.

3.4.1 Tổng quan và phân rã chi phí hạ tầng

Số liệu ghi nhận ngày 10/01/2026 cho thấy hệ thống có mức chi phí duy trì thấp, phù hợp với mô hình IoT quy mô lớn cụ thể tổng chi phí tiêu thụ từ tháng 1 đến thời điểm hiện tại là \$2.65. Hệ thống dự báo tổng chi phí cho cả tháng vận hành liên tục 24/7 với 4 trạm quan trắc sẽ rơi vào khoảng \$7.30 tăng nhẹ so với mức \$3.86 của tháng 12/2025 tăng tần suất gửi dữ liệu để phục vụ quá trình kiểm thử chịu tải.

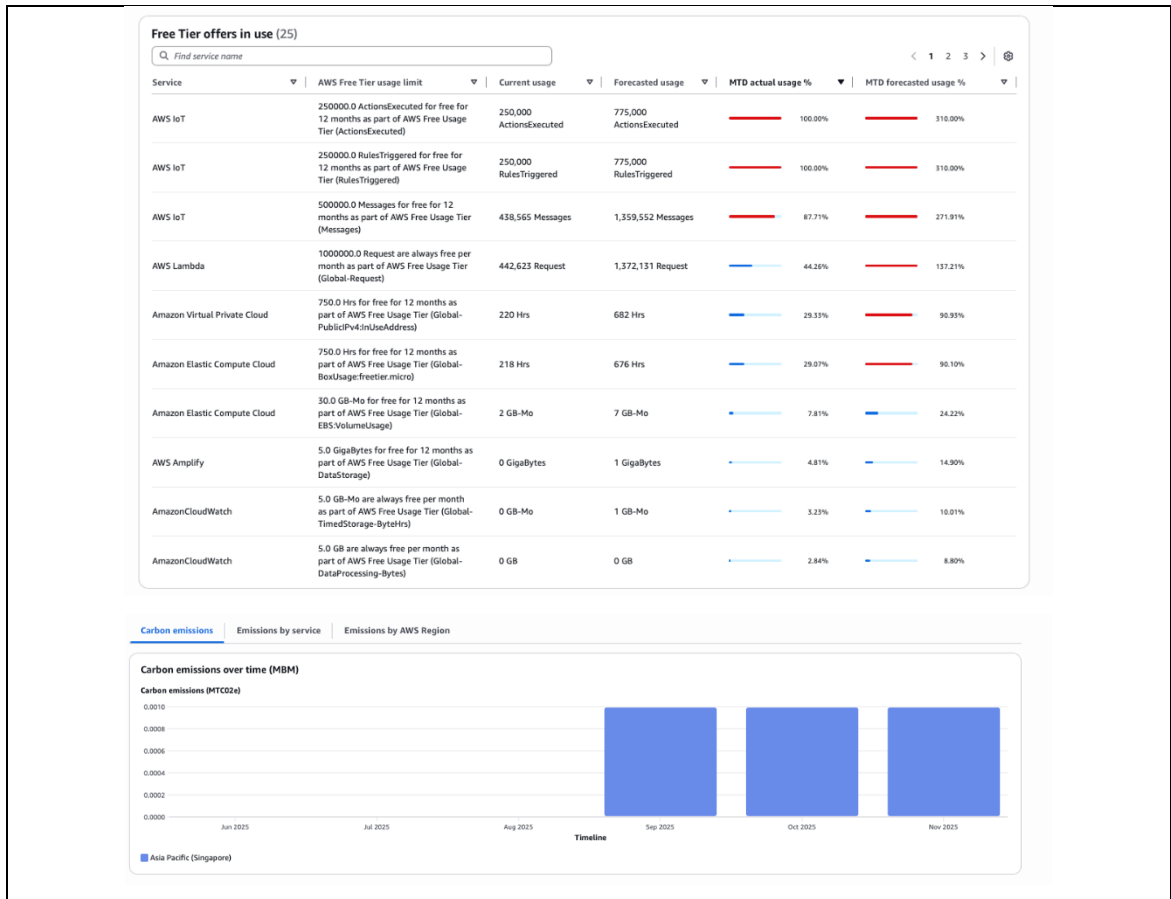
Đi sâu vào cơ cấu chi phí, biểu đồ phân ra dịch vụ tại hình 3.24 chỉ ra rằng Amazon DynamoDB là thành phần chiếm tỷ trọng lớn nhất \$1.6 tương ứng gần 60% tổng chi phí do đặc thù lưu trữ dữ liệu NoSQL lớn và liên tục từ các trạm quan trắc. Tiếp theo là chi phí cố định cho dịch vụ tên miền Route 53 khoảng \$0.50 và chi phí truyền nhận tin nhắn qua AWS IoT Core khoảng \$0.30. Việc xác định rõ các nguồn phát sinh chi phí là cơ sở quan trọng đến đề xuất các phải pháp tối ưu hoá trong tương lai cụ thể như chuyển đổi dữ liệu sang dạng lưu trữ lạnh AWS S3 Glacier để giảm tải cho DynamoDB.



Hình 3.17. Tổng quan chi phí và biểu đồ phân ra dịch vụ

3.4.2 Đánh giá mức độ sử dụng tài nguyên và tác động môi trường

Đề tài đã tận dụng hiệu quả gói ưu đãi AWS Free Tier dành cho nhà phát triển, tuy nhiên quá trình thử nghiệm hệ thống với cường độ cao cũng bộc lộ những giới hạn.



Hình 3.18. Báo cáo sử dụng Free Tier và khí thải carbon

Theo báo cáo sử dụng tài nguyên hình 3.18, dịch vụ AWS IoT Core đã chạm ngưỡng 100% hạn mức miễn phí cho số lượng tác vụ thực thi kèm theo dự báo cuối tháng có thể

vượt qua mức 310% nếu giữ nguyên tần suất. Ngược lại, dịch vụ tính toán AWS Lambda vẫn nằm trong vùng an toàn với mức sử dụng 44.26%.

Bên cạnh đó, hướng tới tiêu chuẩn hệ quan trắc xanh - GreenIoT thì hệ thống cũng được đánh giá về mặt tác động khí thải carbon. Báo cáo AWS ghi nhận lượng khí thải ước tính cực thấp chỉ với 0.001 MTCO₂e trong suốt 3 tháng vận hành. Điều này cho thấy kiến trúc serverless không chỉ tiết kiệm chi phí mà còn thân thiện với môi trường hơn đáng kể so với việc duy trì các máy chủ vật lý truyền thống.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN MỞ RỘNG

Kết luận

Sau quá trình nghiên cứu và thực hiện đề tài “Tối ưu code nhúng cho bộ điện tử của hệ đo liều phóng xạ và nâng cấp web hiển thị theo thời gian thực” đã hoàn thành các nội dung và mục tiêu đã đề ra. Xây dựng thành công hệ thống quan trắc theo tinh thần full-stack từ các công tác phần cứng, phần mềm firmware đến các kỹ thuật liên quan đến back-end trên nền tảng AWS và front-end tạo ra ra web.

Về mặt tối ưu hoá mã nguồn nhúng, đề tài đã làm chủ kỹ thuật lập trình trên vi điều khiển ESP32, phân tách thành công luồng xử lý mạng và luồng đo đặc cảm biến. Các giải thuật nâng cao như ngắt phần cứng với IRAM_ATTR và cơ chế tự phục hồi đã được thực hiện, đảm bảo hệ thống hoạt động chính xác không bỏ sót xung tín hiệu và có khả năng tự động khôi phục khi gặp sự cố mất kết nối.

Về mặt nâng cấp hệ thống Web và hạ tầng AWS, đề tài đã chuyển đổi thành công từ mô hình giám sát thụ động sang mô hình tương tác hai chiều. Giao diện Web Dashboard được thiết kế lại chuyên nghiệp với khả năng vẽ biểu đồ độ trễ thấp, tích hợp các công cụ phân tích dữ liệu lịch sử và giám sát chi phí vận hành. Đặc biệt phân hệ quản trị với các tính năng điều khiển từ xa như cập nhật phần mềm OTA, khởi động lại thiết bị và cấu hình tham số mạng, giúp tối ưu hoá quy trình bảo trì và vận hành hệ thống trên diện rộng với chi phí thấp khoảng \$7.30/tháng cho 4 trạm quan trắc.

Hướng phát triển mở rộng hệ thống

Trên cơ sở nền tảng phần cứng và phần mềm đã được tối ưu, đề xuất các hướng phát triển tiếp theo nhằm hoàn thiện sản phẩm hướng tới thương mại hoá.

Thứ nhất về khả năng kết nối với năng lượng. Hệ thống cần được nghiên cứu tích hợp các chuẩn truyền dữ liệu diện rộng năng lượng thấp như NoRaWAN hoặc NB-IoT để mở rộng phạm vi hoạt động tới các khu vực không có sóng WiFi. Đồng thời việc tối ưu hoá sâu hơn các chế độ ngủ kết hợp với mạch sạc năng lượng mặt trời sẽ là giải pháp then chốt để gia tăng chính cơ động và thời gian hoạt động độc lập của thiết bị.

Thứ hai về trải nghiệm người dùng và xử lý dữ liệu. Hướng phát triển tiếp theo là xây dựng ứng dụng di động đa nền tảng để cung cấp khả năng cảnh báo tức thời qua thông báo đẩy. Bên cạnh đó việc tích hợp các mô hình trí tuệ nhân tạo vào quy trình xử lý dữ liệu đám mây sẽ giúp hệ thống không chỉ dừng lại ở việc hiển thị số liệu mà còn sở hữu khả năng tự động phát hiện bất thường và dự báo xu hướng an toàn bức xạ một cách thông minh nhất.

TÀI LIỆU THAM KHẢO

- [1] P. N. N. Minh, Thiết lập hệ thống truyền dữ liệu AWS thời gian thực tuyến ứng dụng cho hệ quan trắc phóng xạ, TP. Hồ Chí Minh, 2024.
- [2] N. Kolban, "Kolban's Book on ESP32," 2018. [Online]. Available: <https://leanpub.com/kolban-ESP32>.
- [3] O. Standard, "MQTT Version 5.0," 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- [4] A. W. Services, "AWS IoT Core Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/iot/>.
- [5] A. W. Services, "AWS Amplify Documentation - Build and deploy full-stack apps," 2024. [Online]. Available: <https://docs.aws.amazon.com/amplify/>.
- [6] E. Systems, "ESP32 Technical Reference Manual," 2020. [Online]. Available: <https://www.espressif.com/>.
- [7] E. Systems, "Over The Air Updates (OTA) API Guide," ESP-IDF Programming Guide, 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/ota.html>.
- [8] A. W. Services, "Analyzing your costs with AWS Cost Explorer," 2024. [Online]. Available: <https://docs.aws.amazon.com/cost-management/latest/userguide/ce-what-is.html>.

PHỤ LỤC 1 - MÃ NGUỒN CÁC TRẠM QUAN TRẮC PHÓNG XẠ

config_globals.h

```
#pragma once

// --- 1. THƯ VIỆN ---
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include "DHT.h"
#include <time.h>
#include <NTPClient.h>
#include <ArduinoOTA.h>
#include <EEPROM.h>
#include <SPI.h>
#include <TFT_eSPI.h>
#include "secrets.h"
#include <math.h>

#define DHT11Pin 5
#define DHTType DHT11
#define PIN_TICK 27

#define CONVERSION_FACTOR 151.0
#define AWS_IOT_PUBLISH_TOPIC
"/tram/pub"
#define AWS_IOT_SUBSCRIBE_TOPIC
"/tram/sub"

// Ngưỡng cảnh báo
#define WARNING_THRESHOLD 1.0
#define DANGER_THRESHOLD 5.0

DHT HT(DHT11Pin, DHTType);
TFT_eSPI tft = TFT_eSPI();
TFT_eSprite spr = TFT_eSprite(&tft);
WiFiUDP ntpUDP;
WiFiClientSecure net;
NTPClient timeClient(ntpUDP,
"vn.pool.ntp.org");
PubSubClient client(net);

static float humi = 0;
static float tempC = 0;
static long timeTick = 0;
const char *statusStation = "Tram";
int addr = 0;
int lastSignalState = HIGH;
portMUX_TYPE timerMux =
portMUX_INITIALIZER_UNLOCKED;
static volatile unsigned long counts = 0;
static float uSv;
static float cps = 0;
static float backgroundCPS = 0.0;
static unsigned long int timeCount_present;

static unsigned long int count_present;
static unsigned long int timeCount_prev = 0;
static unsigned long int count_prev = 0;
static String address = "227 - Nguyen Van Cu";
static float wr_ctrl = 0;
static float wr = 0;
uint64_t messageTimestamp;
unsigned long lastMillis = 0;
unsigned long lastPublishMillis = 0;
const int publishInterval = 7000;
const int measureInterval = 1000;
unsigned long previousMillis = 0;
int timeDelay = 7000;
StaticJsonDocument<500> StationDoc;
StaticJsonDocument<500> TempDoc;

time_t now;
time_t nowish = 1510592825;
unsigned long epochTime;

// KHAI BÁO CÁC HÀM
void Init();
void eeprom();
void arduinoOTA();
void connectAWS();
void readData();
void display();
void tube_impulse();
void publishMessage();
void tempEvent();

void countPulses() {
    int currentSignalState =
digitalRead(PIN_TICK);
    if (lastSignalState == HIGH &&
currentSignalState == LOW) {
        counts++;
    }
    lastSignalState = currentSignalState;
}

//sensors
void IRAM_ATTR tube_impulse(void) {
    portENTER_CRITICAL_ISR(&timerMux);
    counts++;
    portEXIT_CRITICAL_ISR(&timerMux);
}

//aws-mqtt
unsigned long getTime()
{
    struct tm timeinfo;
    if (!getLocalTime(&timeinfo))
    {
        return (0);
    }
    time(&now);
    return now;
}

void NTPConnect(void)
{
    Serial.print("Setting time using SNTP");
    configTime(7 * 3600, 0 * 3600, "pool.ntp.org",
"time.nist.gov");
    now = time(nullptr);
    while (now < nowish)
    {
        delay(500);
        Serial.print(".NTPConnect.");
        now = time(nullptr);
    }
    Serial.println("done!");
    struct tm timeinfo;
    gmtime_r(&now, &timeinfo);
    Serial.print("Current time: ");
    Serial.print(asctime(&timeinfo));
}

void messageReceived(char *topic, byte
*payload, unsigned int length)
{
    Serial.print("Received [");
    Serial.print(topic);
    Serial.print("]: ");
    for (int i = 0; i < length; i++)
    {
        Serial.print((char)payload[i]);
    }
    Serial.println();
}

void connectAWS() {
    delay(3000);
    WiFi.mode(WIFI_STA);
    NTPConnect();

    net.setCACert(AWS_CERT_CA);
    net.setCertificate(AWS_CERT_CRT);
    net.setPrivateKey(AWS_CERT_PRIVATE);

    client.setServer(AWS_IOT_ENDPOINT,
8883);
    client.setCallback(messageReceived);
    Serial.println("Connecting to AWS IOT");
}
```

```

while (!client.connect(THINGNAME)) {
    if (millis() - lastMillis > timeDelay * 10) {
        lastMillis = millis();
        Serial.print(".connectAWS.");
        delay(1000);
        ESP.restart();
    }
}
if (!client.connected()) {
    Serial.println("AWS IoT Timeout!");
    return;
}

client.subscribe(AWS_IOT_SUBSCRIBE_TOPIC);
Serial.println("AWS IoT Connected!");
}

void publishMessage()
{
    StaticJsonDocument<200> doc;
    timeClient.update();
    unsigned long epochTime =
timeClient.getEpochTime();
    unsigned long localEpochTime = epochTime;

    time_t now = time(nullptr);

    struct tm timeinfo;
    localtime_r(&now, &timeinfo);

    char buffer[32];
    strftime(buffer, sizeof(buffer), "%c",
&timeinfo);

    doc["LocalTime"] = buffer;
    doc["timestamp"] = now;
    doc["humidity"] = humi;
    doc["temperature"] = tempC;
    doc["uSv"] = uSv;
    doc["cps"] = cps;
    doc["counts"] = count_present;
    doc["stationName"] = String(statusStation);

    char jsonBuffer[512];
    serializeJson(doc, jsonBuffer);
    client.publish(AWS_IOT_PUBLISH_TOPIC,
jsonBuffer);
}

//ota-wifi
void Init() {
    Serial.begin(115200);
    EEPROM.begin(512);

    tft.init();
    #ifndef TFT_BL
        pinMode(TFT_BL, OUTPUT);
        digitalWrite(TFT_BL, HIGH);
    #endif
    tft.setRotation(1);
    spr.createSprite(tft.width(), tft.height());

    spr.fillRect(TFT_BLACK);
    spr.setTextColor(TFT_GREEN,
TFT_BLACK);
    spr.setTextDatum(MC_DATUM);
    spr.drawString("Khoi Dong...", tft.width() / 2,
tft.height() / 2);
    spr.pushSprite(0, 0);

    HT.begin();
    pinMode(PIN_TICK, INPUT);
    // Gắn ngắt an toàn
    attachInterrupt(digitalPinToInterrupt(PIN_TICK)
, tube_impulse, FALLING);

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    Serial.print("Connecting WiFi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi Connected");

    lastMillis = millis();
}

void eeprom()
{
    {
        if (char(EEPROM.read(addr)) == 't') {
            EEPROM.write(addr, 'f');
            EEPROM.commit();
        } else {
            if (WiFi.status() != WL_CONNECTED) {
                EEPROM.write(addr, 't');
                EEPROM.commit();
                ESP.restart();
            }
            Serial.println("WiFi connected (EEPROM
check)");
        }
    }
}

void arduinoOTA()
{
    ArduinoOTA.setHostname(statusStation);
    ArduinoOTA.setPasswordHash("123456789");
    ArduinoOTA.onStart([]()
    {
        String type;
        if (ArduinoOTA.getCommand() ==
U_FLASH) {
            type = "sketch";
        } else {
            type = "filesystem";
        }
        Serial.println("Start updating " + type);
        ArduinoOTA.onEnd([]()
        { Serial.println("\nEnd"); });
        ArduinoOTA.onProgress([](unsigned int
progress, unsigned int total)
        { Serial.printf("Progress:
%u%%\r", (progress / (total / 100))); });
        ArduinoOTA.onError([](ota_error_t error)
        {
            Serial.printf("Error[%u]: ", error);
            if (error == OTA_AUTH_ERROR) {
                Serial.println("Auth Failed");
            } else if (error == OTA_BEGIN_ERROR) {
                Serial.println("Begin Failed");
            } else if (error == OTA_CONNECT_ERROR)
{
                Serial.println("Connect Failed");
            } else if (error == OTA_RECEIVE_ERROR) {
                Serial.println("Receive Failed");
            } else if (error == OTA_END_ERROR) {
                Serial.println("End Failed");
            }
        });
    }

    //peripherals
    void changedname(const char *message, size_t
length)
    {
        DynamicJsonDocument doc(1024);
        deserializeJson(doc, message);
        address = String(doc["namec"]);
        Serial.println("address");
    }

    void storagebysec(const char *message, size_t
length)
    {
        wr = 0;
        Serial.print("storagebysec");
    }

    void storageby1min(const char *message, size_t
length)
    {
        wr = 1;
        Serial.print("storageby1min");
    }
}

```



```

void storageby5min(const char *message, size_t
length)
{
    wr = 5;
    Serial.print("storageby5min");
}
void connectEvent(const char *payload, size_t
length)
{
    StationDoc["id"] = "";
    Serial.println("Station ESP connectEvent");
    serializeJson(StationDoc, Serial);
    char msg[256];
    serializeJson(StationDoc, msg);
}
void tempEvent()
{
    TempDoc["address"] = address;
    TempDoc["uSv"] = uSv;
    TempDoc["cps"] = cps;
    TempDoc["humi"] = round(humi);
    TempDoc["tempC"] = round(tempC);
    TempDoc["counts"] = count_present;
    TempDoc["statusStation"] =
String(statusStation);
    TempDoc["date"] = timeCount_present;
    TempDoc["realtime"] = timeCount_present;
    TempDoc["wr_contl"] = wr_contl;
    TempDoc["wr"] = wr;
    TempDoc["a"] = 5;
    char msg[256];
    serializeJson(TempDoc, msg);
}

void display() {
    if (uSv >= DANGER_THRESHOLD) {
        spr.fillSprite(TFT_RED);
        spr.setTextColor(TFT_WHITE);
        spr.setTextDatum(MC_DATUM);
        if ((millis() / 500) % 2 == 0) {
            spr.setTextSize(3);
            spr.drawString("DANGER", tft.width() /
2, tft.height() / 2 - 20);
        }
        spr.setTextSize(2);
        spr.drawFloat(uSv, 3, tft.width() / 2,
tft.height() / 2 + 30);
    } else if (uSv >= WARNING_THRESHOLD) {
        spr.fillSprite(TFT_ORANGE);
        spr.setTextColor(TFT_BLACK);
        spr.setTextDatum(MC_DATUM);
        spr.setTextSize(3);
        spr.drawString("WARNING", tft.width() / 2,
tft.height() / 2 - 20);
        spr.setTextSize(2);
        spr.drawFloat(uSv, 3, tft.width() / 2,
tft.height() / 2 + 30);
    } else {
        spr.fillSprite(TFT_BLACK);
        spr.setTextDatum(TL_DATUM);
        spr.setTextSize(2);
        spr.setTextColor(TFT_WHITE);
        char buffer[32];
        sprintf(buffer, "T: %.1f C", tempC);
        spr.drawString(buffer, 15, 10);
        sprintf(buffer, "H: %.1f %%", humi);
        spr.drawString(buffer, 180, 10);
        spr.drawFastHLine(0, 40, tft.width(),
TFT_DARKGREY);
        spr.setTextColor(TFT_ORANGE);
        spr.setTextDatum(TR_DATUM);
        spr.drawFloat(uSv, 3, 305, 65, 4);
        spr.setTextDatum(TL_DATUM);
        spr.setTextSize(2);
        spr.drawString("uSv/h", 15, 60);
        spr.setTextColor(TFT_YELLOW);
        spr.setTextDatum(TR_DATUM);
        spr.drawFloat(cps, 2, 305, 115, 4);
        spr.setTextDatum(TL_DATUM);
        spr.setTextSize(2);
        spr.drawString("CPS", 15, 110);
        spr.drawFastHLine(0, 155, tft.width(),
TFT_DARKGREY);
        portENTER_CRITICAL(&timerMux);
        unsigned long dispCounts = counts;
        portEXIT_CRITICAL(&timerMux);
        spr.setTextSize(1);
        spr.setTextColor(TFT_CYAN);
        spr.drawString("Tong dem: " +
String(dispCounts), 15, 160);
        time_t now_disp = time(nullptr);
        struct tm* ptm = localtime(&now_disp);
        strftime(buffer, sizeof(buffer),
"%H:%M:%S", ptm);
        spr.drawString(buffer, 260, 160);
    }
    spr.pushSprite(0, 0);
}

void readData() {
    unsigned long currentMillis = millis();

    // Tính toán mỗi 1 giây
    if (currentMillis - lastMillis >=
measureInterval) {
        float elapsedSeconds = (currentMillis -
lastMillis) / 1000.0;

        portENTER_CRITICAL(&timerMux);
        unsigned long currentCounts = counts;
        portEXIT_CRITICAL(&timerMux);

        float deltaCounts = (float)(currentCounts -
count_prev);
        cps = deltaCounts / elapsedSeconds;
        uSv = (cps * 60.0) /
CONVERSION_FACTOR;

        float newHumi = HT.readHumidity();
        float newTemp = HT.readTemperature();
        if (!isnan(newHumi)) humi = newHumi;
        if (!isnan(newTemp)) tempC = newTemp;

        // Logic điều khiển lưu trữ (Trạm 1)
        timeClient.update();
        unsigned long epoch =
timeClient.getEpochTime();
        if (wr == 0) wr_contl = 0;
        else if (wr == 1) wr_contl = (epoch % 60 ==
0) ? 0 : 1;
        else if (wr == 5) wr_contl = (epoch % 300
== 0) ? 0 : 1;

        tempEvent();
        display();

        count_prev = currentCounts;
        lastMillis = currentMillis;
    }

    // Gửi AWS (Mỗi 7 giây)
    if (currentMillis - lastPublishMillis >
publishInterval) {
        if (!client.connected()) {
            connectAWS();
        }
        client.loop();
        publishMessage();
        lastPublishMillis = currentMillis;
    }
}

```

Trạm 1

46

```
CJ98JI3R911sMgqCLcl73S+Hci66lKHdgSf//5R
4/9wtNTI93zp/rCg=
-----END RSA PRIVATE KEY-----
)KEY";
```

Trạm 2

```
#include <pgmspace.h>
#define SECRET
const char WIFI_SSID[] = "hcmus-CPL";
//TAMIM2.4G
const char WIFI_PASSWORD[] =
"0838301986";
#define THINGNAME
"ESP8266_Station_2"
int8_t TIME_ZONE = 7; //NYC(VN): +7
UTC
const char AWS_IOT_ENDPOINT[] =
"a1wbkazcw8mwk5-ats.iot.ap-southeast-
1.amazonaws.com";
// Amazon Root CA 1 and 3
static const char AWS_CERT_CA[]
PROGMEM = R"EOF(
-----BEGIN CERTIFICATE-----
MIIDQTCCAimgAwIBAgITBmyfz5m/jA
o54vB4ikPmljZbyjANBgkqhkiG9w0BAQ
sF
ADA5MQswCQYDVQVQGEwJVUzEPM
A0GA1UEChMGQW1hem9uMRkwFwY
DVQQDExBBbWw6
b24gUm9vdCBDQSAxMB4XDTE1MDU
yNjAwMDAwMFoXDTE4MDEwNzAw
MDAwMFowOTEL
MAKGA1UEBhMCVVMxMDEwNzAwNzAw
AoTBkFTYXpwbjEzMDEwNzAwNzAw
W1hem9uIjFv
b3QgQ0EgMTCCASlwDQYJKoZIhvcNA
QEBBQADggEPADCCAQoCggEBALJ4
gHHKeNXj
ca9HgFB0fW7Y14h29Jlo91ghYPl0hAEvr
AlthtOgQ3pOsqTQNroBvo3bSMgHFzZ
M
9O6II8c+6zfltrN4SWiw3te5djgdYZ6k/oI
2peVKVuRF4fn9tBb6dNqcmzU5L/qw
IFAGbHrQgLKm+a/sRxmPUDgH3KKH
OVj4utWp+UhnMJbulHheb4mjUcAwhma
hRWa6
VOujw5H5SNz/0egwLX0tdHA114gk957
EWW67c4cX8jJGKLhD+redqsq08p8kDi1
L
93FcXmn/6pUCyziKrlA4b9v7LWlbcce
VOF34GfID5yHI9Y/QCB/IIDEgEw+OyQ
m
jgSubJrIlg0CAwEAAANCMEEAwDwYD
VR0TAQH/BAUwAwEB/ZAQOBgNVHQ8
BAf8EBAMC
```

```
AYYwHQYDVR0OBBYEFiQYzIU07Lw
MIJQuCFmcx7lQTgoIMA0GCSqGSIb3D
QEBCwUA
A4IBAQCqY8jdaQZChGsV2USggNiMOru
You6r4lK5lpDB/G/wkjUu0yKGX9rbxen
DI
U5PMCCjmmCXPI6T53iHTfIUJrU6adTrC
C2qJeHZERxhlbI1Bjlt/msv0tadQ1wUs
N+gDS63pYaACbvXy8MWy7Vu33PqU
XHeeE6V/Uq2V8viTO96LXFvKWlJbYK
8U90vv
o/ufQJVtMVT8QtPHRh8jrdkPSHCa2XV
4cdFyQzR1bldZwgJcJmApzyMZFo6lQ6
XU
5MsI+yMRQ+hDKXJioaldXgjUkK642M
4UwtBV8ob2xJNDd2ZhwLnoQdeXcGA
Dbkpy
rqXRfboQnoZsG4q5WTP468Sqvvg5
-----END CERTIFICATE-----
)EOF";
static const char AWS_CERT_CRT[]
PROGMEM = R"KEY(
-----BEGIN CERTIFICATE-----
MIIDWTCCAKGgAwIBAgIUlYIkj3lrlQ4
LJWzAFP5LK8hmKIwwDQYJKoZIhvcN
AQEL
BQAwTTFtLMEkGA1UECwxwCQW1hem
9uIFdlYiBTZXJ2aWNlcYBPPUftYXpvi
5jb20g
SW5jLiBMPVNIYXR0bGUgU1Q9V2Fza
GluZ3RvbiBDPVVtMB4XDTE1MDkyN
TA3MzEz
M1oXDTQ5MTIzMTIzNTk1OVowHjEc
MBoGA1UEAwwTQVtTIElVVCBDZXJl
0aWZpY2F0
ZTCCASlwDQYJKoZIhvcNAQEBBQAD
ggEPADCCAQoCggEBAPjJl56gjfPO8bW
COVcAf
kUaIquXCd1G846av+RK+WnoBkcAv+E
00UmjheK5w/XSZlr6CsfLrtgN9PeDheop
9
5jk12kIbGFZ9twzT3V+WoqtEaDdrQJK2
x6N2FH5szTpJ2ntUfcXsyCrt6H3kIMq+
Qy9e+nFcd3RAT8mSns8ZF07Qg6i9Uc8
qedtCE/9RM9NmiXZ6FREnEv9vDO/774
3
8pVgDeieQgGZB/7TEvuwd+WgdjPg3lgI
uSelE7uf/43dZpjh0b1Tr3vhTbTVQkb
```

```
GrDJ/3R8Nzt/RObyCG4qFFecR2og/hhBk
BGh7evYj3qF1gwIu65NWjrWzQgnRqw
O
DbUCAwEAAANgMF4wHwYDVR0jBBBg
wFoAUUV/Xoi/jfIFp0qHMQgkppMiVC2SE
wHQYD
VR0OBBYEFHhGdDMbDjd3h8n3iN1FC
2qBJ3r7MAwGA1UdEwEB/wQMAAAw
DgYDVR0P
AQH/BAQDAgeAMA0GCSqGSIb3DQE
BCwUAA4IBAQBGMi/KbQz0Y28Xqut9
+wgOkt4n
KYrQzkHbmTX0o764WWKUcWWRIOD
w45PohsR+Oyhu7D5TlkcWeSpaCsc+e2U
eTJyS
js0ic0yKeC8qWw/gT1J1Z5g5+8Au310fn
Si/4zGNSJtMyRRSLtCNnphw6WS7Lk/K
+9aM/k+CWvg8cZslygFJNT+FH2bWdN
MnjtN2czrhRZhlbnmRjbr7nevVTmM6yO
Ai
7wSBcuJWxXnzXNRKRpPQlPqZewQWq
5Hg3U8jwlex1Wab12OnTOozf5QJTyj/7l
q+
dNAsMjGirqct+4j24JiTwcm4IjgE8PdBJ
WfRCPO7efh9qU7NUMvU49r87G9
-----END CERTIFICATE-----
)KEY";
// Device Private Key
//change this
static const char AWS_CERT_PRIVATE[]
PROGMEM = R"KEY(
-----BEGIN RSA PRIVATE KEY-----
MIIEpgIBAAKCAQEA+OXnqCN887xtY
I5VwB+RRoiq5cJ3Ubzjpq/5Er5aegGRwC
/4
TTRSaOF4rnD9dJmWvoKx8uu2A3094O
Fyin3mOSXaQhsYVn23DNPdX5Y6p8Ro
N2tA
krbHo3YUfminZNOknae1R9xezIKu3ofeQg
yr5DL176cVyd3dEBPyZKexzkXTtCDqL
l
Rzyxp20IT/1Ez02aJdnoVESc+/28M7/vvjf
ylWAN6J5CAZKh/tMS+7AP5aB2OmDe
WAi5J6UTu0l//jd1mmOHRvVOve+FMG
1BWRsasMn/dHw3O39E5vIlbioU5xHai
D+
GEGQEAHt69iPeoXWDAi7rk1aOtbNCCd
GrA4NtQIDAQABAOIBAQChtv39URqZ
ViEK
```

N1UYZitFV/kfGHHr50FvFPnT1aCpNR
vvXyxx0eqqqN/USQUit8AbPRxNaZxvAj
j
Q/7oawqaprOgc/Ur98RmDKR2wqZa3S
CVyt+bFNUMYT953yy3V2i+frGFCq8Y
yDL
9t0sg9orRaCg3JwmkvUpcZvDXCqovlqG
xSxzNiGasic0COsfXTmzZAhw2rStQpun
oDzkP5yyDXILLDmwZivn16DVGfHg5v
WJ19vXzWeMxp4vNv8u/rBQHPrgdG8Q
wolb
vltW1eENDqPwx5P/nWjvw3f0AEDuqq6/
yIz5GNaNpsi3lweKaRBhByqKakX92m
MQ
43Y0ZJNdaAoGBAP6g7Wj24Z08SHNPfN
uxdf/AOv7oz6XvQc8DQED+4JOHv43niu
VN

pn2nly1+FmGG4oi5w/0LFcQjQFXK5QF
BbNgw6zHBEyHN2sWAXWNqYywyM
gAtVhyb
XNeKzEh97b4fZdVVDXJUvaDPbOtT99
HypMuwxNqwkjVObs/mmJvdg3IzAoGB
APo9
E5Hm2R4aPzFt+N5yLfvIU/lyEvWejUOb
e24EyF2Fv6P/9eKijXbvgYcBn17iHyPa
was/CEByh++uOqHp3kflQGo6YoMUV2j
PVq/1q/0sSNCnj6zuQZi73oiost8qQ3sa
GYW08iveuVUxgc8LRMNkCbCGG8CIs
cYKBXLpdCh3AoGBANBW7kofDtUveP
xYgonj
dvAu4lamORjFXiW6tb04jppHzTUXye77
DQntMWY4r6lWy04h6TnHcyTLzx0jaS
MQ
3DU8OYS5PKVBIQrOkBsC9mgi1+6LK
dU4TevStnluCDOyzwixkdIFgi7s6WrXho
MT

LUG4vilNpiPvRBeRzzZhFec/AoGBAPhi
BXnN9PbPQKh0qxFJrmD78HZqB22ryjY
w
L6z9rFBAQP+tQkiJ5vcGNsBmFhsxUY2
KRrAO6cbmqA19GA+9Dni4kl6ew0euuj
B7
lWUk6mlrOn95wr/44j2TRQjKwlnDvqW
DO5Afej3/XmC0uk9jtQZo1sFT7FollMS
p
D82G8lhZAoGBAl/h3T3GN6XYVIOkVC
PhyDmoe/hvANnolBYFPtPnB19me56Bm
XQx
GkSw/alREkud96TVsOdJC9NzUOAeBlm
fBGItHND5JVZT6NDcgk0Ycq3lOnnpKk
YD
T5ZgLOOjwIulfoCCY3Z3U5t2PtDJEa+g
il8vVI97KSKHrOrJIdm0Iujf
-----END RSA PRIVATE KEY-----
)KEY";

Trạm 3

```
#include <pgmspace.h>
#define SECRET
const char WIFI_SSID[] = "hcmus-CPL";
//TAMIM2.4G
const char WIFI_PASSWORD[] =
"0838301986";
#define THINGNAME
"ESP8266_Station_3"
int8_t TIME_ZONE = 7; //NYC(VN): +7
UTC
const char AWS_IOT_ENDPOINT[] =
"a1wbkazcw8mwk5-ats.iot.ap-southeast-
1.amazonaws.com";
// Amazon Root CA 1 and 3
static const char AWS_CERT_CA[]
PROGMEM = R"EOF(
-----BEGIN CERTIFICATE-----
MIIDQTCCAimgAwIBAgITBmyfz5m/jA
o54vB4ikPmljZbyjANBgkqhkiG9w0BAQ
sF
ADA5MQswCQYDVQQGEwJVUzEPM
A0GA1UEChMGQW1hem9uMRkwFwY
DVQQDExBBbWF6
b24gUm9vdCBDQSAxMB4XDTE1MDU
yNjAwMDAwMFoXDTE4MDEwNzAw
MDAwMFowOTEL
MAkGA1UEBhMCVVMxMDEwNzAwNjB
AoTBKkFYXpjbEJZMBGA1UEAxMQQ
W1hem9uIHFv
b3QgQ0EgMTCCASlwDQYJKoZIhveNA
QEBBQADggEPADCCAQoCggEBALJ4
gHHKKeNXj
```

ca9HgFB0fW7Y14h29Jl091ghYP10hAEvr
AlthtOgQ3pOsQTNroBvo3BmSGHFzZ
M
9O6I18c+6zfltrn4SWiw3te5djgdYZ6k/ol
2peVKVuRF4fn9tBb6dNqcmzU5L/qw
IFAGbHrQgLKm+a/sRxmPUDgH3KKH
OVj4utWp+UhnMJbulHheb4mjUcAwhma
hRWa6
VOujw5H5SNz/0egwLX0tdHA114gk957
EWW67c4cX8jJGKLhD+redqsq08p8kDi1
L
93FcXmn/6pUCyziKrlA4b9v7LWlbcce
VOF34Gf1D5yHI9Y/QCB/IIDEgEw+OyQ
m
jgSubJrlqg0CAwEAAaNCMEAwDwYD
VR0TAQH/BAUwAwEB/zAOBgNVHQ8
BAf8EBAMC
AYYwHQYDVR0OBBYEFiQYzIU07Lw
MIJQuCFmcx7lQTgo1MA0GCSqGSIb3D
QEBCwUA
A4IBAQCY8jdaQZChGsV2USggNiMOru
You6r4IK5IpDB/G/wkjUu0yKGX9rbxen
DI
U5PMCCjjmCXPI6T53iHTfIUJrU6adTrC
C2qJeHZERxhlb1Bjjt/msv0tadQ1wUs
N+gDS63pYaACbvXy8MWy7Vu33PqU
XHeeE6V/Uq2V8viTO96LXFvKWlJbYK
8U90vv
o/ufQJVtMVT8QtPHRh8jrdkPSHCa2XV
4cdFyQzR1bldZwgJcJmApzyMZFO6IQ6
XU

5Msl+yMRQ+hDKXJioaldXgjUkK642M
4UwtBV8ob2xJNDd2ZhWLnOQdeXeGA
Dbkpy
rqXRfboQnoZsG4q5WTP468SQvvG5
-----END CERTIFICATE-----
)EOF";
static const char AWS_CERT_CRT[]
PROGMEM = R"KEY(
-----BEGIN CERTIFICATE-----
MIIDWTCCAKGgAwIBAgIUcHff0Fb7m
96YcplvBehyLcMo/RAwDQYJKoZIhvcN
AQEL
BQAwTTFLEMEkGA1UECwxQW1hem
9uIFdlYiBTZXJ2aWNleYBPPUFTyXpvi
5jb20g
SW5jLiBMPVNIYXR0bGUgU1Q9V2Fza
GluZ3RvbiBDPVVTMB4XDTE1MTAxN
zA0NTAx
NVoXDTQ5MTIzMtIzNTk1OVowHjEc
MBoGA1UEAwwTQVdTIElvVCBBDZXJ
0aWZpY2F0
ZTCCASlwDQYJKoZIhveNAQEBAQAD
ggEPADCCAQoCggEBAAoitD4uLzNYM
vLj1vNfb
HSoy8+HnSH0hpx2J92LghJTkM1HeeRfy
EpAizNGusJT35tjXGkBhJdKdhnbUzUh
TkYCaIiGPaW5aSlqNDOBZGLY/Yk9vSe
8KjKJmmv3UDAvjmszVv8nOc0x89umX
Ap9
jpOXu4yr4EVSPEMLx6gsJkKwKc9afTx
AFp8n1lAw0p2gxL1R4uf4iQFLXFUXNT
mm


```
V0ujw5H5SNz/0egwLX0tdHA114gk957
EWW67c4cX8jJGKLhD+redqsq08p8kDi1
L
93FcXmn/6pUCyziKrlA4b9v7LWlBxcce
VOF34GfID5yHI9Y/QCB/IIDEgEw+OyQ
m
jgSubJrIqg0CAwEAAaNCMEAwDwYD
VR0TAAQH/BAUwAwEB/zAObgNVHQ8
BAf8EBAMC
AYYwHQYDVR0OBBYEFIQYzIU07Lw
MIJQuCFmex7lQTgoIMA0GCSqGS1b3D
QEBECwUA
A4IBAQCZY8jdaQZChGsV2USggNiMOru
You6r4IK5IpDB/G/wkju0yKGX9rbxen
DI
U5PMCCjImCXPI6T53iHTfIUJrU6adTrC
C2qJeHZERxhlbl1BjIt/msv0tadQ1wUs
N+gDS63pYaACbvXy8MWy7Vu33PqU
XHeeE6V/Uq2V8viTO96LXFvKWlJbYK
8U90vv
o/ufQJVtMVT8QtPHRh8jrdkPSHCa2XV
4cdFyQzR1bldZwgJcJmApzyMZFo6IQ6
XU
5Msl+yMRQ+hDKXJioaldXgjUkK642M
4UwtBV8ob2xJNDd2ZhwLnoQdeXcGA
Dbkpy
rqXRfboQnoZsG4q5WTP468SQvvG5
-----END CERTIFICATE-----
)EOF";
static const char AWS_CERT_CRT[]
PROGMEM = R"KEY(
-----BEGIN CERTIFICATE-----
MIIDWTCCAKGgAwIBAgIUUVQAAtmpkR
2Oecccmaa4e/XpUfMa3UwDQYJKoZIhvc
NAQEL
BQAwwTTLMEkGA1UECwxCQW1hem
9ulFdIYiBTZXJ2aWNleYBPPUFtYXpvbi
5jb20g
SW5jLiBMPVNIYXR0bGUgU1Q9V2Fza
GluZ3RvbiBDPVVtMB4XDTI1MTAyO
DA0NDkx
NFoXDTQ5MTIzMTIzNTk1OVowHjEc
MBoGA1UEAwwTQVdTIElvVCBBDZXJ
0aWZpY2F0
ZTCCASlwDQYJKoZIhvcNAQEBBQAD
ggEPADCCAQoCggEBAN999YW27rlw
+nVIUsBR
IfLGqVj1k/G16nDXOY0UKJIw6qhOxMd
JWKU8kIrROYjgk+6X9dv8/vUDKUdt2
SM
MN4nKAGx0cVPWGEhvjYv5J5sEmxGn
9Zk1ur2n6MFv7zb/EZiCgCVyTm7xYjBS
Mqb
```

```
bQfpLQnukabHnMcos6HPXnCf7iWltm2
vhwwMcbfFfr3aKuT0CYBCt/DTdVmcde
w1
JpXx+cFB4Rx8adgEM5tWE2jx8V/5p2rm
w3o7OJdV0h6aCu7XklBmD/i/TWbd1K8
Z
70OdlkR9lwiYgZ0fSfMqBj3dS3n1CZRn
DArf090eMGI+CTI9axTv3/u41LvTdkKU
eu0CAwEAAaNgMF4wHwYDVR0jBBg
wFoAUoIWbwmAc4lGrE4hFDhZuFrpgT/
wwHQYD
VR0OBBYEFM0uIaCi2jYwcOaxEY2DX
2nNMAKMAwGA1UdEwEB/wQCMAA
wDgYDVR0P
AQH/BAQDAgeAMA0GCSqGS1b3DQE
BCwUAA4IBAQAAn1KEKW38B+uOUQ
B67spw/hbzo
yCIG/WzVrT9AE3+tp0HPfhg81370GhY
ZhRv/y21lky4osoR3BjW3Ju7B0ULmzFq
Y
pvTf2hlcRguqtHrefqV1P2FZ/aIvy/6G55w
rEAsXJlpoOzle2+fg1c8ujIBVT5f4
qbTfnB08XofSaZqVILLD5GTW9xlH3h+
QcOLWJ1W1E+0qnr6tzqIm36sMFB19f3Z
l
A4m0Bc/v1OTzJSKc2LXXc2DTrMRjgG
z2T51tJErt2ijECTo4lRV1NkYM5f6tOqFx
Q
GlkX2oYo3JFZaZlPsW6yfbRb7q7F51z2
YFuPFYiDqzY5ooPgRc9sUG2qf1le
-----END CERTIFICATE-----
)KEY";

// Device Private Key
//change this
static const char AWS_CERT_PRIVATE[]
PROGMEM = R"KEY(
-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEA3331hbbsujD6dU
hSwFGV8sapWPWT8bXqcNc5jRQokjDq
qE7E
x0lYpTyQishE5iOCT7pf12/z+9QMpR23
ZIww3icoAbHrXU9YYSGOBi/knmwSBe
af
1mTW6vafowW/vNv8RmlKAJXJOvFi
MFlypttB+ktCe6Rpsecxyizoc9ecJ/uJYi2
ba+HDAxxt8V+vdoq5PQJgEK38NN1WZ
x17DUmlfH5wUHhHHxp2B4zm1YTaPH
xX/mn
aubDejs4l1XSHpoK7teSUGYP+L9NZt3U
rxns452WRH2XCJiBnR9J8yoGPd1LefUJ
lGcMcT/Tr34wYj4JOX1rFO/f+7jUu9N2
QpR67QIDAQABAoIBAAQj0y586wMG
AT22
```

```
70F8wWhT4cMyGWF62zLn6bsBSpcarN
4dR2vhofC9pHfNqekRc1KCw0+8VSqvD
OTq
HgGQzz7uPsF5R8d95sc11emrbXK50cW
LsV5JU+naU66zZo/bC1tKaaEFJUsafVhB
LFYEdwqUeS8b6jrKq87MpDe1Hiy74NJ
GltDcZgki3WSaDegNaEX+hC+AdvhJAjz
h
1fv4/KwWUzShHKy6m/PZ0/dxWIoZgi8
4Nv0XX8oRXjDssh0WYwVNz5mnl2qlP
Gj6
Xl1Md0G1zT8mL+QHhXwkXiTAd3NT
ElhOAjtsW6Z79d3B40VtJq6vYwprCJwZ/
hjj
bN5Ds8ECgYEA9KEdb8/VRc9sNBF9V
OWRmRgtYwSnc++3AhpglfC/aJEXwIJR
pRWf
fdBUjZLlktKW2Mt2/JAHQIXKzxZVV
yysFi7qZA1WY0lZ/QqGIWbQmkOFD7D
XQa
RmpjZy47c9OrmwNt2566zFhxqEy/d7b/23
K6XvHI4a9KTXSDWhyJs8UCgYEA6eF
T
2YATAGLQoUtNHgNyQPNo/Bfv7+svU
YfCVUsc7yMCBFOJeaKja7j3qEvBYbY
G09Si
AKCFv1iA7ZE3gDtXTpUHo1sHDxUlur
RUljN0GBCT8jSUCAC8VzCoAOYfB7R
QEdCq
6i2OE/p9QLX1EWFZJew64ZUCFa1Obw
5EWtfmFQkCgYEAxYmolJZ2ONYjIPSS
iVCL
vt1YISIr+iGzXT5hdbQfvtETH3/cX/tS/Cy
TanMCBejiW0OLoIlZHllhi2dDF/w
vytkMSQNLIVISXKc8rGE+a2Ub2ltNM
FOKFGPZaGqaRebJw/RN4rJ0TeEyJOZm
Re
vR0qw+podV2wGVWhj82FoKUCgYBUb
JMg+xtg00quPola4TGYAoWoinHpMAN
HsfLT
aSbLp95tg14nuy60jvNPG/F1fQif5KyJ/2f
oN2Fq4DWho0oTwrYcIJq+tgqFh9zU
z0QD/8/36UTaV9incpfioy7mUA9rLuDT
YZhe29nkxwnsuTA99af3kBTNid67NfI0
vKSASQKBgHqiM9uVVplu9nR9l1eYbh
0FtWOT3XUOpwRly+m6BDIOvs/8KM
EAiJ0l
kn9bwxsuCEuHE3dtQ81d0xovYVuktUR
XCCBiLVY226icnfyzkwkBYGXZ/0rj9w
vu
7LNYhsZWnN1k0c8M321tFESi7nA8ILO
V6KVvwqlfuJp60/F+AAW
-----END RSA PRIVATE KEY-----
)KEY";
```